CRIME CASE CENTRAL

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y **Christian Miguel Giménez Puerma**





Anteproyecto Base.

Denominación del proyecto	Crime Case Central (central de casos criminales).
Integrantes el grupo: (máximo 3)	Christian Miguel Giménez Puerma y Ahmed Hassan Khamis.
Justificación del proyecto. (por qué pensáis que es interesante)	Una aplicación web para gestionar crímenes, cubriendo la necesidad de centralizar y optimizar la información delictiva. Esta herramienta permitirá a las fuerzas estado, criminólogos y detectives dar de alta, modificar y eliminar registros de crímenes con detalles precisos, facilitando la rápida recuperación de información clave. La capacidad de listar, ordenar, buscar y filtrar crímenes mejorará la eficacia en la investigación, promoviendo la colaboración entre entidades encargadas de la seguridad y fortaleciendo los esfuerzos de aplicación de la ley.
Descripción técnica y tecnologías que utilizar para el desarrollo de este.	Este proyecto adoptará una arquitectura LAMP (Linux, Apache, MySQL, PHP) con Laravel en el backend y JavaScript Vanilla en el frontend. Laravel será la pieza central para gestionar crímenes mediante un sistema CRUD, utilizando MySQL para la sección de persistencia de datos. JavaScript se encargará de tareas específicas como validación de datos, y eventos varios como el filtrado y la presentación de la tabla de crímenes, contribuyendo así al *look and feel* de la

Proyecto 2ºDAW Anteproy
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

	aplicación. La interfaz de usuario se diseñará de manera intuitiva y eficiente, centrándose en la experiencia del usuario y utilizando JavaScript para mejorar la interactividad y el aspecto visual sin sobrecargar el frontend. El despliegue se llevará a cabo en un entorno Apache.
Reparto del trabajo entre los integrantes del grupo. (Poner las horas estimadas para cada una de las partes de trabajo).	En total especulamos que van a ser como mínimo las 80 horas disponibles, repartidas en 40 horas cada uno. Puede variar dependiendo de las circunstancias de los alumnos. • Planificación: 5h. • Diseño: 10h. • Implementación: 30h. • Pruebas: 10h. • Despliegue: 10h. • Documentación: 10h. Ambos integrantes trabajarán equitativamente en cada una de las secciones del proyecto.
Observaciones:	Este proyecto en principio apunta a ser ambicioso, pero con tiempo y experimentación puede que se reduzcan algunas partes del objetivo final. Esto se debe a que queremos asegurarnos de entregar algo que cumpla con unos mínimos de calidad.
A rellenar por parte del equipo docente	
Observaciones:	
Apto:	**No Apto:**

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

ÍNDICE

1. Project Introduction (english)
1. Introducción del proyecto (español)
    Descripción de los contenidos del proyecto asociados a los módulos del C.F.G.S. D.A.W.
    Objetivos medibles
        Objetivo principal
            Gestión de crímenes
        Objetivos extras (añadidos tras alcanzar el objetivo principal)
            Gestión de usuarios
            Gestión de incidencias
            Control de permisos y seguridad de acceso
        Objetivos específicos
            Crear crímenes
            Editar crímenes
            Eliminar crímenes
            Consultar crímenes
        Objetivos específicos extras (añadidos tras alcanzar los objetivos específicos principales)
            Crear incidencias
            Listar Incidencias
    Finalidad
2. Análisis del entorno
    Descripción de oportunidades de negocio
    Obligaciones fiscales, laborales y de prevención de riesgos del proyecto.
        Obligaciones Fiscales:
        Obligaciones Laborales:
        Obligaciones de Prevención de Riesgos:
    Posibles ayudas o subvenciones para el desarrollo del proyecto.
3. Análisis del sistema actual
4. Solución propuesta
    Diferentes tecnologías existentes para el desarrollo y puesta a punto de la aplicación que se desarrollará.
    Estudio económico. Valoración de presupuestos.
        1. Presupuesto Actual:
        2. Costos de Infraestructura:
        3. Implementación de la API de Google Maps:
        4. Aumento de la Capacidad del Servidor:
        5. Presupuesto Propuesto:
    Elección y justificación de las tecnologías seleccionadas.
    Recursos humanos y materiales necesarios para desarrollar el proyecto.
5. Planificación
    Metodología de trabajo

Proyecto 2º DAW. Autores:
Khaled Hassan Khamis
Cristian Miguel Giménez Puerma

Fases del desarrollo (Extreme Programming)
Análisis de los problemas de las aplicaciones actuales.
Entrevistas con el cliente para obtener las especificaciones del proyecto.
Necesidades del cliente:
Restricciones de acceso:
Ciberseguridad:
Objetivo del proyecto:
Diseño de la solución a adoptar.
Interfaz de usuario:
Gestión de datos delictivos:
Sistemas de búsqueda:
Restricciones de acceso:
Ciberseguridad:
Redacción de los presupuestos.
Adquisición del software y hardware.
Medios a utilizar
Medios Hardware
Medios Software
Desarrollo de la aplicación.
1. Diseño de la Arquitectura:
2. Selección de Tecnologías:
3. Desarrollo de Funcionalidades:
4. Integración de la API de Google Maps:
5. Pruebas:
6. Seguridad:
7. Implementación y Despliegue:
8. Mantenimiento y Actualizaciones:
Fase de pruebas.
Formación de los usuarios finales.
Lista de modificaciones o ampliaciones futuras que se podrían aplicar a la aplicación proyectada
Plan de mantenimiento de la aplicación.
Mantenimiento Regular:
Actualizaciones de Seguridad:
Actualizaciones Funcionales:
Pruebas de Regresión:
Documentación:
Formación de Usuarios:
Soporte al Usuario:
6. Estudio de la viabilidad del proyecto
7. Documentación del diseño e implementación de la solución adoptada.
Prototipo de la aplicación. Diseño de interfaces.
Diseño lógico de la aplicación a desarrollar.
Diagrama UML

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Cristian Miguel Giménez Puerma

Descripción modular del software a desarrollar:
Diagrama de flujo del aplicativo:
Diagrama casos de uso:
Caso de uso Usuario tipo Lector
Caso de uso Usuario tipo Editor
Caso de uso Usuario tipo Moderador
Caso de uso Usuario tipo Administrador/Super Administrador
Diseño de la base de datos:
Diagrama Entidad-Relación
Diagrama Relacional
Justificación del no uso del ORM Eloquent de Laravel
Otras estructuras de datos que se utilizan en la aplicación.
Estudio de la seguridad de la aplicación.
8. Código fuente documentado
Código Enrutador
Fichero enrutamiento web.php
Código Middlewares
Fichero middleware EnsureTokenIsValid.php
Fichero middleware ValidarPermisos.php
Código Controladores
Fichero CrimenController.php
Método crear()
Método editar()
Métodos de obtención de datos: get_crimen(), get_tipologias(), datos_editar(), datos_leer()
Método borrar()
Fichero UsuarioController.php
Método entrar()
Método crear()
Método editar()
Métodos de obtención de datos: get_usuario() y datos_leer()
Método borrar()
Fichero IncidenciaController.php
Fichero BuscadorController.php
Código módulos de JavaScript
Fichero crimenes.js
Fichero usuarios.js
Fichero incidencias.js
Fichero menu.js
Fichero validarFormularios.js
Fichero dragNdrop.js
9. Manual de configuración y funcionamiento
Instalación de LAMP
Configurar site y pruebas

Proyecto 2º DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Configuración de permisos
Instalación de Laravel
Configuración de Apache para el correcto enrutamiento
Configuración del site de Apache
10. Manual de usuario
11. Plan de Formación a los Usuarios de la Aplicación
Contenidos a tratar.
Tiempo estimado de impartición de cada contenido.
12. Bibliografía utilizada para cada contenido.
Documentación criminóloga, legislación y fondos
Otras bibliografías (miscelánea)

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

1. Project Introduction (english)

Crime Case Central is an innovative web application that seeks to revolutionize the way crime-related information is handled. Designed with a focus on efficiency and accessibility, this platform offers a variety of tools and functionalities designed to facilitate the management of criminal data. With an intuitive and easy-to-use interface, Crime Case Central is intended to meet the needs of both law enforcement professionals and criminology experts.

In addition to managing crime records in a precise and detailed manner, Crime Case Central incorporates search systems that allow users to easily locate specific information. The application also has permission systems and access restrictions, ensuring that only authorized users can access certain sections and modules, thus protecting the confidentiality and integrity of the data.

Cybersecurity and session control is an integral part of the platform's security, with mechanisms that detect and respond to unauthorized access attempts. In case of failed access attempts, the application is designed to automatically block the user after a specific number of erroneous attempts, which helps to prevent unauthorized access.

Proyecto 2ºDAW. Autores:
Ahmed Hassan Rebaine
y Christian Miguel Giménez Puerma

With these features, Crime Case Central promises to significantly improve the way criminal cases are addressed and investigation is conducted, providing robust security and more efficient and secure data management.

1. Introducción del proyecto (español)

Crime Case Central es una aplicación web innovadora que busca revolucionar la manera en que se maneja la información relacionada con crímenes. Diseñada con un enfoque en la eficiencia y la accesibilidad, esta plataforma ofrece una variedad de herramientas y funcionalidades diseñadas para facilitar la gestión de datos delictivos. Con una interfaz intuitiva y fácil de usar, Crime Case Central está destinada a satisfacer las necesidades tanto de profesionales encargados de hacer cumplir la ley como de expertos en criminología.

Además de gestionar registros de crímenes de manera precisa y detallada, Crime Case Central incorpora sistemas de búsqueda que permiten a los usuarios localizar información específica con facilidad. La aplicación también cuenta con sistemas de permisos y restricciones de acceso, asegurando que solo los usuarios autorizados puedan acceder a ciertas secciones y módulos, protegiendo así la confidencialidad y la integridad de los datos.

La ciberseguridad y el control de sesión es una parte integral de la seguridad de la plataforma, con mecanismos que detectan y responden a intentos de acceso no autorizados. En caso de intentos de acceso fallidos, la aplicación está diseñada para bloquear automáticamente al usuario después de un número específico de intentos erróneos, lo que contribuye a prevenir el acceso no autorizado.

Con estas características, Crime Case Central promete mejorar significativamente la forma en que se abordan los casos criminales y se lleva a cabo la investigación, proporcionando una seguridad robusta y una gestión de datos más eficiente y segura.

Descripción de los contenidos del proyecto asociados a los módulos del C.F.G.S. D.A.W.

Módulo del Grado Superior	Conocimientos aplicados en el proyecto
Sistemas informáticos	Instalación y configuración de los entornos de desarrollo y producción en Ubuntu, conexión por SSH al servidor, uso de comandos Linux y creación de un script bash para el despliegue.
Bases de datos	Uso de MySQL para la persistencia de datos y creación de esquemas E-R y Relacional.
Programación	Implementación de la lógica de programación y de negocio tanto en el Frontend como en el Backend.
Lenguajes de marcas y sistemas de gestión de información	Uso de CSS y HTML en el frontend así como el intercambio de datos con ficheros JSON.
Entornos de desarrollo	Desarrollo de la aplicación web Crime Case Central en el IDE Visual Studio Code, utilización de repositorio remoto GIT y creación de diagramas de flujo y UML.
Desarrollo web en entorno cliente	Creación de módulos JavaScript para la realización de ordenamiento de tablas, validación de formularios, sistema Drag & Drop, peticiones AJAX y eventos e interacción entre el usuario y la interfaz gráfica.
Desarrollo web en entorno servidor	Uso y redacción de estructura en 3 capas, utilizando el lenguaje de programación PHP en la lógica de negocio del Backend, así como la investigación y aplicación del framework Laravel.
Despliegue de aplicaciones web	Creación de manual de despliegue, uso e instalación de arquitectura LAMP, realizando la pertinente configuración de Apache y configurando los entornos de desarrollo y producción (VPS local).
Diseño de interfaces web	Creación de la guía de estilos UI/UX, diseño de la interfaz gráfica con la herramienta Figma, implementación de

	los estilos de las vistas generadas en HTML con CSS, realizando una interfaz responsive que se adapte para dispositivos desktop/tablet/mobile.
Formación y orientación laboral	Redacción de la prevención de riesgos, obligaciones fiscales y laborales, y un plan de formación detallado a los usuarios.
Empresa e iniciativa emprendedora:	Redacción del plan de empresa en la memoria con sus respectivos gastos para el proyecto, así como el cumplimiento de las normativas de Protección de Datos y Privacidad (L.O. 5/3/2018), y la Ley de Responsabilidad Medioambiental en España (Ley 26/2007).
Inglés técnico	Lectura y aprendizaje de las distintas tecnologías en las webs oficiales en inglés, así como redacción de la introducción de la aplicación en inglés, usando palabras técnicas aprendidas durante el curso.

Objetivos medibles

Los objetivos medibles de este proyecto constan de los siguientes puntos:

Objetivo principal

Gestión de crímenes

La aplicación debe administrar una base de datos de crímenes para permitir a los usuarios crear, leer, editar y eliminar crímenes. Esto facilitará la gestión centralizada y proporcionará una solución de software a criminólogos, detectives, periodistas y fuerzas del estado.

Objetivos extras (añadidos tras alcanzar el objetivo principal)

Gestión de usuarios

La aplicación debe administrar una base de datos de usuarios para darlos de alta, editar sus datos y borrarlos, estableciendo un sistema jerárquico de roles y permisos que vuelva la aplicación más robusta y segura.

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Gestión de incidencias

La aplicación debe administrar una base de datos de incidencias para permitir a los usuarios moderadores informar sobre datos erróneos, equivocados y falsedades a los usuarios administradores.

Control de permisos y seguridad de acceso

La aplicación debe asegurar la confidencialidad, integridad y el correcto uso tanto de las funcionalidades de la misma, como sobre todo de los datos sensibles almacenados en ella, permitiendo que sólo los usuarios con el permiso adecuado vean y accedan a determinadas secciones así como bloqueando el acceso a aquellos usuarios no identificados.

Objetivos específicos

Crear crímenes

Permitir el registro de crímenes con imágenes adjuntas, nombre, descripción, etc.
Implementar un sistema de validación de datos para garantizar la precisión y veracidad de la información ingresada al registrar crímenes, incluyendo la detección de posibles errores o duplicaciones en los datos proporcionados.

Editar crímenes

Brindar la capacidad de modificar crímenes previamente creados, incluyendo la posibilidad de agregar nuevos valores a los ya definidos.
Desarrollar una interfaz de usuario intuitiva y fácil de usar que permita a los usuarios editar los detalles de los crímenes de manera eficiente, pudiendo además sustituir las imágenes de archivo ya alojadas.

Eliminar crímenes

Proporcionar la opción de eliminar los crímenes existentes.
Implementar un mecanismo de eliminación de crímenes que garantice la integridad de la base de datos, permitiendo la gestión adecuada de las relaciones de datos para evitar inconsistencias en el sistema, así como eliminando todas las imágenes y datos relevantes del crímen.

Consultar crímenes

Permitir la lectura y visualización de imágenes y detalles de los crímenes almacenados. Integrar funcionalidades de búsqueda que permitan a los usuarios

consultar crímenes de manera eficiente, utilizando criterios como ubicación geográfica, tipo de crimen, fecha, entre otros, para obtener resultados precisos y relevantes.

Objetivos específicos extras (añadidos tras alcanzar los objetivos específicos principales)

Crear incidencias

Facilitar el registro de incidencias, permitiendo a los usuarios moderadores informar de ellas mediante un botón accesible y reconocible.

Listar Incidencias

Permitir la visualización y consulta de las incidencias registradas, ofreciendo opciones para ordenar la lista según diferentes criterios/columnas. así como un acceso intuitivo a las acciones a realizar con las mismas.

Borrar Incidencias

Proveer una función para eliminar incidencias existentes en el listado de las mismas.

Crear Usuarios

Implementar un procedimiento para la creación de nuevos usuarios, incluyendo la recopilación de datos esenciales como nombre, permisos y fecha de creación entre otros.

Listar Usuarios

Habilitar una interfaz para mostrar un listado de usuarios, con herramientas para ordenar, editar y borrar los mismos.

Editar Usuarios

Ofrecer la capacidad de modificar la información de usuarios existentes, con una interfaz clara y sencilla que facilite el proceso.

Borrar Usuarios

Establecer un mecanismo para la eliminación segura de usuarios, asegurando que no se afecte la integridad de los datos relacionados, haciéndolo de manera accesible en el listado de los mismos.

Buscar Crímenes

Integrar un sistema de búsqueda que permita a los usuarios encontrar crímenes específicos, buscando en los campos de los mismos para poder filtrar según convenga.

Buscar Usuarios

Integrar un sistema de búsqueda que permita a los administradores encontrar usuarios específicos, buscando en todos los campos de los mismos para poder filtrar según convenga.

Buscar Incidencias

Implementar herramientas de búsqueda para que los administradores puedan filtrar y encontrar incidencias de manera eficiente.

Bloquear Intentos de Acceso Fallidos

Configurar un sistema de seguridad que limite el número de intentos de acceso fallidos (10 intentos fallidos resultan en 2 horas de bloqueo).

Restringir Acceso a Secciones y Módulos

Desarrollar un control de acceso que impida a los usuarios entrar a secciones y módulos para los cuales no tienen permisos.

Proyecto 2DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Finalidad

Crime Case Central es una aplicación web diseñada para ofrecer una solución integral en la gestión de crímenes, centrándose en la necesidad de facilitar la administración y optimización de la información delictiva. Esta herramienta permite a las fuerzas del estado, criminólogos y detectives realizar operaciones clave como dar de alta, modificar y eliminar registros de crímenes con precisión y eficacia. Al brindar la capacidad de listar, ordenar, buscar y filtrar crímenes, Crime Case Central promueve la rápida recuperación de información relevante, lo que a su vez mejora significativamente la eficiencia en la investigación criminal. Además, al fomentar la colaboración entre las entidades encargadas de la seguridad, la aplicación fortalece los esfuerzos de aplicación de la ley, contribuyendo así a la resolución efectiva de casos y al mantenimiento del orden público.

2. Análisis del entorno

Descripción de oportunidades de negocio

En el paisaje digital actual, la necesidad de acceso rápido y confiable a información sobre crímenes y delitos es más apremiante que nunca. Nuestra plataforma surge como una respuesta innovadora a esta demanda creciente, ofreciendo un archivador centralizado de crímenes y consultas relacionadas, diseñado para proporcionar a usuarios de diversos ámbitos una herramienta esencial en la toma de decisiones y en la gestión de riesgos.

Ante el creciente desafío de centralizar y optimizar los datos relacionados con crímenes, esta plataforma proporciona una solución eficaz para las fuerzas del estado, criminólogos y detectives. Con las funcionalidades anteriormente citadas, la aplicación aprovecha las oportunidades de negocio emergentes en el campo de la seguridad y la justicia. Además, al promover la colaboración entre entidades encargadas de la seguridad, la aplicación se posiciona como una herramienta clave para fortalecer los esfuerzos de aplicación de la ley y mejorar la eficacia en la investigación criminal.

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Sebastián Negrín Cabrera Cuerma

Obligaciones fiscales, laborales y de prevención de riesgos del proyecto.

Durante el desarrollo y la operación de nuestro proyecto, Crime Case Central, nos comprometemos a cumplir con una serie de obligaciones fiscales, laborales y de prevención de riesgos para garantizar el cumplimiento normativo y el bienestar de nuestros empleados y clientes.

Obligaciones Fiscales:

Para asegurar una conducta fiscal responsable y transparente, nos comprometemos a:

- Registrarse adecuadamente como empresa y obtener el número de identificación fiscal (NIF) correspondiente.

- Cumplir con todas las regulaciones fiscales locales relacionadas con impuestos sobre ingresos, ventas y servicios.
- Implementar un sistema de facturación electrónica que cumpla con los estándares fiscales vigentes.
- Presentar declaraciones fiscales periódicas y anuales, reflejando nuestros ingresos, gastos y otros aspectos financieros de manera precisa y oportuna.

Obligaciones Laborales:

Nos comprometemos a mantener relaciones laborales justas y equitativas, cumpliendo con las siguientes obligaciones:

- Seguir las leyes laborales locales en cuanto a contratación, despidos, horas de trabajo, salario mínimo y condiciones laborales.
- Inscribir a nuestros empleados en el sistema de seguridad social y cumplir con los pagos de contribuciones correspondientes.
- Garantizar un ambiente de trabajo seguro y saludable, cumpliendo con las normativas de seguridad e higiene laboral.
- Proporcionar formación y capacitación adecuadas a nuestros empleados en materia de seguridad laboral y prevención de riesgos.

Obligaciones de Prevención de Riesgos:

Para asegurar la seguridad y el bienestar de nuestros empleados y clientes, nos comprometemos a:

- Realizar una evaluación exhaustiva de riesgos laborales para identificar y mitigar los peligros potenciales en el lugar de trabajo.
- Elaborar un plan de prevención de riesgos laborales que incluya medidas específicas para mitigar los riesgos identificados.
- Proporcionar el equipamiento de protección personal necesario y garantizar su uso adecuado por parte de nuestros empleados.
- Proporcionar formación continua en prevención de riesgos laborales para mantener a nuestro equipo informado y preparado para actuar en caso de emergencia.

Estas obligaciones son fundamentales para asegurar la integridad, la sostenibilidad y el éxito de nuestro proyecto, así como para cumplir con nuestros compromisos éticos y legales como empresa responsable en el mercado.

Posibles ayudas o subvenciones para el desarrollo del proyecto.

Nuestro fondo para el desarrollo de este proyecto consta de nuestro aporte personal de socios y de una inversión en criptomonedas. Además del capital sugerido, el proyecto tratará de apoyarse en la subvención pública de Incentivos y ayudas para emprendedores y autónomos del Servicio Público de Empleo Estatal (SEPE), el cual ofrece información sobre reducciones y bonificaciones a la Seguridad Social, bonificación a trabajadores por cuenta propia o autónomos por conciliación de la vida profesional y familiar vinculada a contratación, Ayudas ICO, y ayudas de las Comunidades Autónomas.

3. Análisis del sistema actual

La funcionalidad de la aplicación ha sido detallada en secciones anteriores. Sin embargo, es importante abordar el riesgo potencial asociado con el filtrado de información hacia personas no autorizadas durante el uso por parte del cliente final. Para mitigar este riesgo, implementaremos medidas que garanticen un control efectivo sobre el registro de usuarios, tanto en su alta como en su baja. De esta manera, aseguraremos que todo el proceso se lleve a cabo dentro de un perímetro controlado y seguro.

Las necesidades que conlleva la aplicación involucran a empresas que requieran gestionar este proceso integralmente. Por lo tanto, será necesario contar con instalaciones adecuadas, como oficinas, para llevar a cabo la gestión de usuarios y la infraestructura necesaria para ejecutar la aplicación de manera efectiva.

4. Solución propuesta

Diferentes tecnologías existentes para el desarrollo y puesta a punto de la aplicación que se desarrollará.

El desarrollo de la aplicación web Crime Case Central consta de una arquitectura LAMP con Laravel 10 y el uso de Blade, JavaScript Vanilla, AJAX, CSS y HTML en el frontend, así como Laravel para la lógica de negocio y MySQL para la persistencia de datos, disponiendo así de una arquitectura robusta y flexible.



Presentación: Los templates Blade proporcionados por Laravel junto con HTML, CSS y JavaScript proporcionan una interfaz de usuario rica y dinámica. Blade ofrece un sistema de plantillas intuitivo que facilita la integración con el backend de Laravel, mientras que JavaScript Vanilla y AJAX se utilizan para mejorar la interactividad y la experiencia del usuario sin la sobrecarga de frameworks adicionales.

Lógica de Negocio: Laravel 10 en el backend actúa como el núcleo de la aplicación, gestionando la lógica de negocio y las interacciones con el servidor web Apache y el intérprete mod_php. Laravel ofrece una estructura clara y convenciones

que ayudan a mantener el código organizado y coherente, aparte de brindar un sistema de enrutamiento seguro, la posibilidad de añadir Middlewares para el control de acceso a la aplicación y el uso de la arquitectura MVC (Modelo Vista Controlador) que permite la modularización necesaria del aplicativo para un mejor mantenimiento y desarrollo.

Persistencia de Datos: El SGBD MySQL maneja la capa de persistencia de datos, almacenando y recuperando información de manera eficiente. El filesystem Storage de Laravel proporciona una interfaz para gestionar los archivos de imágenes necesarios para el aplicativo, lo que permite un manejo de archivos y datos coherente y seguro, recuperando de la base de datos las rutas pertinentes de los archivos en el servidor.

Estudio económico. Valoración de presupuestos.

1. Presupuesto Actual:

El presupuesto actual para el proyecto Crime Case Central es de 30 € mensuales. Este presupuesto se asigna principalmente a la infraestructura, que está alojada en un servidor privado virtual (VPS) anónimo por razones de seguridad.

2. Costos de Infraestructura:

La infraestructura actualmente se aloja en un servidor privado virtual (VPS) anónimo. Este tipo de alojamiento proporciona un alto nivel de seguridad y privacidad, lo cual es esencial para una aplicación que maneja datos sensibles como Crime Case Central. El costo de este alojamiento está incluido en el presupuesto mensual actual de 30 €.

3. Implementación de la API de Google Maps:

La implementación de la API de Google Maps en el proyecto requerirá un aumento en el presupuesto debido a los costos asociados con el uso de esta API. Los primeros 200 dólares de uso mensual son gratuitos, pero después se aplican tarifas. Para un uso moderado de la API, se estima un costo adicional de alrededor de 20 € al mes.

4. Aumento de la Capacidad del Servidor:

Además, es posible que necesitemos aumentar la capacidad de nuestro servidor para manejar las solicitudes adicionales a la API de Google Maps. Esto podría incrementar el costo de la infraestructura en unos 10 € al mes.

5. Presupuesto Propuesto:

Teniendo en cuenta estos factores, proponemos aumentar el presupuesto mensual a 60 €. Este aumento cubrirá los costos adicionales asociados con la implementación de la API de Google Maps y el aumento de la capacidad del servidor, asegurando que la aplicación funcione sin problemas y mantenga un alto nivel de seguridad y eficiencia.

Por supuesto, estos son sólo estimaciones y los costos reales pueden variar. Según vayamos gastando los 200 dólares gratuitos de la API de Google Maps, iremos revisando los detalles de la misma para ajustar el presupuesto según sea necesario.

Elección y justificación de las tecnologías seleccionadas.

Como se ha comentado anteriormente, este proyecto adoptará una arquitectura LAMP (Linux, Apache, MySQL, PHP) con Laravel en el backend, y vistas Blade con JavaScript Vanilla en el frontend. Laravel será la pieza central para gestionar crímenes mediante un sistema CRUD, utilizando MySQL para la sección de persistencia de datos. JavaScript se encargará de tareas específicas como validación de datos, y eventos varios como el filtrado, el sistema de Drag & Drop del archivador de imágenes y la presentación de la tabla de usuarios e incidencias, contribuyendo así al look and feel de la aplicación. La interfaz de usuario se diseñará de manera intuitiva y eficiente, centrándose en la experiencia del usuario y utilizando JavaScript para mejorar la interactividad y el aspecto visual sin sobrecargar el frontend. El despliegue se llevará a cabo en un entorno Apache.

La elección del software se basa en los siguientes criterios:

1. **Servidor Apache:** Se ha seleccionado debido a su disponibilidad gratuita, lo que lo convierte en una opción económica.
2. **Lenguaje PHP:** Se escoge por ser de uso libre y no tener costos asociados.

3. **Base de datos SQL**: Se elige por su alto rendimiento y fiabilidad, además de ser de uso gratuito.
4. **Arquitectura Linux:** Se prefiere por su ligereza y eficiencia en el consumo de recursos en comparación con otros sistemas operativos.
5. **Servidor VPS**: Se opta por su accesibilidad a los servidores y la externalización del mantenimiento de hardware, lo que implica costos más bajos en comparación con otros modelos de alojamiento.

Recursos humanos y materiales necesarios para desarrollar el proyecto.

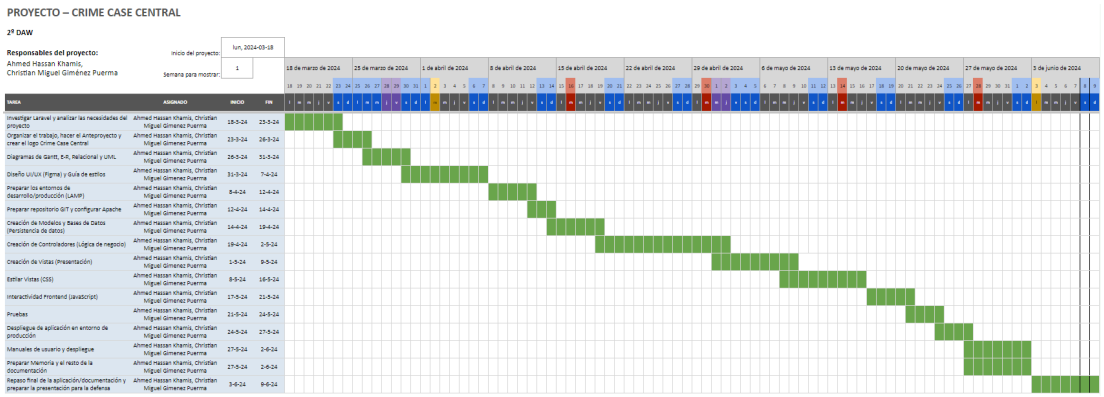
Se empleará un equipo de 800 personas para la gestión de esta organización, distribuidas de la siguiente manera: tres cuartos de este equipo se dedicarán a la lectura y gestión de crímenes, mientras que un cuarto se encargará de aspectos empresariales como recursos humanos, despidos, contratos, reclutamiento y otras tareas relacionadas.

Los materiales necesarios para el desarrollo del proyecto incluyen equipos informáticos para el personal, los cuales deberán contar con el software necesario y gratuito para la ejecución de la aplicación.

Proyecto 2DAW. Autores:
Ahmed Hassan Khamis
Christian Miguel Giménez Puerma

5. Planificación

Diagrama de Gantt con la planificación del desarrollo del proyecto (actualizado a día 17 de Junio de 2024):



Fechas desglosadas para su mejor visualización:

- **18-23 Marzo**
  Investigar Laravel y analizar las necesidades del proyecto.
- **23-26 Marzo**
  Organizar el trabajo, hacer el Anteproyecto y crear el logo Crime Case Central.
- **26-31 Marzo**
  Realizar los Diagramas de Gantt, E-R, Relacional y UML.
- **31-07 Marzo/Abril**
  Diseño UI/UX (Figma) y Guía de estilos.
- **08-12 Abril**
  Preparar los entornos de desarrollo/producción (LAMP).
- **12-14 Abril**
  Preparar repositorio GIT y configurar Apache.
- **14-19 Abril**
  Creación de Modelos y Bases de Datos (Persistencia de datos).
- **19-02 Abril/Mayo**
  Creación de Controladores (Lógica de negocio).
- **01-09 Mayo**
  Creación de Vistas (Presentación).
- **08-16 Mayo**
  Estilar Vistas (CSS).
- **17-21 Mayo**
  Interactividad Frontend (JavaScript).
- **21-24 Mayo**
  Pruebas.
- **24-27 Mayo**
  Despliegue de aplicación en entorno de producción.
- **27-02 Mayo/Junio**
  Manuales de usuario y de despliegue.
  Realizar la Memoria del proyecto y el resto de la documentación.
- **03-09 Junio**
  Repaso final de la aplicación y de la documentación.
  Preparar la presentación para la defensa.

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Metodología de trabajo

Para la realización de este proyecto, empleamos la metodología de desarrollo de software ágil ***Extreme Programming***, enfocándonos en el ***Pair Programming*** (programar de manera coordinada mediante servicios de streaming). El objetivo es evitar posibles fallos de programación y asegurar la mejor calidad posible del software. Proporcionar feedback y retroalimentación fluida y constante, facilitar la alineación y la toma de decisiones conjuntas. Esto permitirá adaptarse a los cambios y responder rápidamente a las necesidades u obstáculos que se enfrenten en el proyecto, así como dividir las tareas cuando sea necesario. La elección de esta metodología se basa en su flexibilidad, comunicación fluida y la capacidad de agilizar el proceso de desarrollo para acotar el escaso tiempo disponible, al mismo tiempo que permite aprender sobre la marcha las nuevas tecnologías requeridas por el proyecto. Simultáneamente al trabajo conjunto de programación, aprender el framework y las nuevas tecnologías necesarias, de modo que se pueda cumplir con los objetivos a tiempo, pudiendo suplir las carencias académicas sin que ello afecte a la calidad del producto final.

Fases del desarrollo (Extreme Programming)

A continuación se exponen las fases por las cuales pasaron los entregables en cada iteración, las cuales se realizarán en las fechas indicadas: 16/04/2024, 30/04/2024, 14/05/2024, 28/05/2024, 14/06/2024 y 17/06/2024.

1. **Planificar**: Establecer los objetivos de la iteración y planificar las sub tareas necesarias para alcanzarlos (to-do).
2. **Diseñar**: Diseñar la solución para los requisitos específicos de esta iteración.
3. **Implementar**: Implementar el diseño en el formato que se solicite.
4. **Pruebas**: Realizar pruebas para garantizar que el entregable cumpla con los requisitos.
5. **Documentar**: Realizar la documentación pertinente a la iteración.
6. **Entregar**: Realizar el envío del entregable solicitado.

Estas actividades se llevan a cabo de manera iterativa y se repiten en cada ciclo de desarrollo en Extreme Programming, las cuales se van a llevar a cabo en cada una de las tareas asignadas en el Diagrama de Gantt, expuestas anteriormente en el apartado de planificación.

Análisis de los problemas de las aplicaciones actuales.

Los problemas de las aplicaciones actuales en la gestión de crímenes incluyen cuestiones de privacidad y protección de datos, especialmente en el contexto del análisis de datos. Además, el riesgo de sesgos algorítmicos y errores en los sistemas de IA podría llevar a conclusiones erróneas o injusticias en los casos criminales. La seguridad en aplicaciones web es fundamental, ya que las vulnerabilidades pueden ser aprovechadas por ciberdelincuentes para acceder a bases de datos y otros datos sensibles.

La aplicación Crime Case Central abordaría estos problemas ofreciendo una plataforma segura para la gestión de crímenes. Algunos puntos clave que justifican la necesidad de esta aplicación son:

Seguridad Mejorada: Que se centra en proteger contra vulnerabilidades comunes y ataques cibernéticos:

- **SQL Injection:** La aplicación evita la inyección SQL mediante el uso de consultas preparadas y parámetros vinculados, lo que garantiza que los datos enviados por los usuarios no puedan ser interpretados como código SQL por el sistema de gestión de bases de datos. Esto previene la manipulación maliciosa de las consultas y protege la integridad de la base de datos.
- **Cross-Site Scripting (XSS):** Se implementan medidas de seguridad como la validación y saneamiento de entradas del usuario, así como la codificación de salida, para evitar que los scripts maliciosos sean inyectados y ejecutados en el navegador de otros usuarios. Esto protege a los usuarios de posibles robos de sesión y otros ataques que comprometan la seguridad del cliente.
- **Cross-Site Request Forgery (CSRF):** Utilizando los tokens anti-CSRF en todos los formularios y peticiones AJAX, la aplicación puede prevenir ataques en los que un sitio malicioso podría realizar acciones no autorizadas en nombre de un usuario autenticado.
- **Session Hijacking:** Con tokens de sesión únicos y la regeneración de estos tras cada inicio de sesión, se reduce el riesgo de que las sesiones de los usuarios sean secuestradas.

- **Inyecciones de archivos:** Al restringir y validar cuidadosamente las cargas de archivos en la funcionalidad de Drag & Drop, se evita que los atacantes suban scripts maliciosos que podrían ejecutarse en el servidor.
- **Ataques de Directorio Transversal:** Los middleware y las restricciones de acceso a archivos aseguran que los atacantes no puedan acceder a directorios fuera de la raíz del servidor web.
- **Fuerza Bruta y Ataques de Diccionario:** El límite de intentos de inicio de sesión y el bloqueo temporal tras fallar múltiples veces, junto con la complejidad requerida en las contraseñas, protegen contra intentos de adivinar credenciales.

Acceso Controlado: Permitiendo solo a entidades y usuarios autorizados el acceso a la información, minimiza el riesgo de acceso no autorizado y uso indebido de los datos:

- **Sistema de Baneo:** La aplicación cuenta con un sistema de baneo que limita los intentos de inicio de sesión fallidos. Un usuario autorizado tiene 10 intentos para iniciar sesión con las credenciales correctas; de lo contrario, será bloqueado del aplicativo durante 2 horas. Esto ayuda a prevenir ataques de fuerza bruta, donde un atacante intenta adivinar contraseñas mediante la prueba y error.
- **Middleware Riguroso:** Dos capas de middleware actúan como guardianes, cortando cualquier tipo de acceso no deseado a rutas o archivos del aplicativo. Si se detecta un intento de acceso no autorizado, el sistema redirige al usuario al login, eliminando posibles accesos no legítimos.
- **Gestión de Permisos:** El uso de tokens únicos y volátiles por usuario y sesión, junto con un control férreo de permisos, asegura que solo los usuarios con la autorización adecuada puedan acceder a áreas restringidas de la aplicación. Esto incluye la restricción de acceso a la zona administrativa para usuarios con menores permisos.

Gestión Eficiente de Datos: Facilitando la organización y el almacenamiento de datos relacionados con crímenes, como imágenes de archivo, fechas de denuncia y tipología del crimen, en un sistema centralizado.

Integridad de la evidencia: Manteniendo un registro detallado de todas las pruebas recolectadas en un caso, desde la fecha de la denuncia hasta el modus

operandi, lo que garantiza la integridad de la evidencia y reduce el riesgo de errores o contaminación.

Cumplimiento de Normativas: Asegurando que la aplicación cumpla con las leyes y regulaciones de protección de datos, lo que es crucial en la gestión de información delicada.

En resumen, Crime Case Central se posiciona como una solución necesaria en el panorama actual, proporcionando una herramienta especializada que atiende a las necesidades específicas de seguridad, gestión y accesibilidad en la documentación y seguimiento de crímenes.

Entrevistas con el cliente para obtener las especificaciones del proyecto.

La entrevista tuvo lugar con un cliente el cual es un profesional encargado de hacer cumplir la ley y está interesado en una solución que facilite la gestión de datos delictivos.

Necesidades del cliente:

El cliente necesita una plataforma que sea eficiente y accesible, con una interfaz intuitiva y fácil de usar. La plataforma debe ser capaz de gestionar registros de crímenes de manera precisa y detallada, y debe incorporar sistemas de búsqueda que permitan a los usuarios localizar información específica con facilidad.

Restricciones de acceso:

El cliente enfatiza la importancia de tener sistemas de permisos y restricciones de acceso para proteger la confidencialidad y la integridad de los datos. Solo los usuarios autorizados deben poder acceder a ciertas secciones y módulos.

Ciberseguridad:

La ciberseguridad y el control de sesión son una parte integral de la seguridad de la plataforma. El cliente menciona que la plataforma debe tener mecanismos que detectan y responden a intentos de acceso no autorizados. En caso de intentos de

acceso fallidos, la aplicación debe bloquear automáticamente al usuario después de un número específico de intentos erróneos, por ejemplo, 10 intentos fallidos deben resultar en 2 horas de bloqueo.

Objetivo del proyecto:

Con estas características, Crime Case Central promete mejorar significativamente la forma en que se abordan los casos criminales y se lleva a cabo la investigación, proporcionando una seguridad robusta y una gestión de datos más eficiente y segura. El cliente está de acuerdo con este objetivo y espera que la implementación de la plataforma pueda ayudar a su organización a lograr estos resultados.

Diseño de la solución a adoptar.

Tras la entrevista con el cliente, el equipo de Crime Case Central propuso la siguiente solución a adoptar:

Interfaz de usuario:

La aplicación contará con una interfaz intuitiva y fácil de usar, diseñada para ser accesible tanto para profesionales encargados de hacer cumplir la ley como para expertos en criminología.

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Gestión de datos delictivos:

La plataforma ofrecerá una variedad de herramientas y funcionalidades para facilitar la gestión de datos delictivos. Esto incluirá la capacidad de gestionar registros de crímenes de manera precisa y detallada.

Sistemas de búsqueda:

Se incorporarán sistemas de búsqueda avanzados que permitirán a los usuarios localizar información específica con facilidad.

Restricciones de acceso:

La aplicación contará con sistemas de permisos y restricciones de acceso para proteger la confidencialidad y la integridad de los datos. Solo los usuarios autorizados podrán acceder a ciertas secciones y módulos.

Ciberseguridad:

Se implementarán mecanismos robustos de ciberseguridad y control de sesión. Estos mecanismos detectarán y responderán a intentos de acceso no autorizados. En caso de intentos de acceso fallidos, la aplicación bloqueará automáticamente al usuario después de un número específico de intentos erróneos.

Con este diseño, Crime Case Central promete mejorar significativamente la forma en que se abordan los casos criminales y se lleva a cabo la investigación, proporcionando una seguridad robusta y una gestión de datos más eficiente y segura.

Redacción de los presupuestos.

Esta tabla muestra el costo mensual de cada componente. La infraestructura (VPS) tiene un costo de 30 € al mes, la API de Google Maps tiene un costo estimado de 20 € al mes y el aumento de la capacidad del servidor tiene un costo estimado de 10 € al mes. El costo total mensual es de 60 €. Por supuesto, estos son sólo estimaciones y los costos reales pueden variar. De cara a futuro,volveremos a revisar los detalles de precios de la API de Google Maps y monitorizaremos el uso para ajustar el presupuesto según sea necesario.

Proyecto 2ºDAW. Autores:

Componente	Costo Mensual (€)
Infraestructura (VPS)	30 €
API de Google Maps	20 €
Aumento de la Capacidad del Servidor	10 €
Total	**60 €**

Adquisición del software y hardware.

Medios a utilizar

Para realizar el proyecto, se van ha utilizar los siguientes recursos:

Medios Hardware

Un servidor virtual propio con 4 núcleos de CPU, 8 GiB de memoria RAM y 50 GiB de memoria secundaria.

Medios Software

IDE Visual Studio Code, Framework Laravel, intérprete de lenguaje PHP, servidor web Apache, Gestor de Base de Datos MySQL, MySQL Workbench, gestor de versiones GIT y sistema operativo GNU/Linux.

Desarrollo de la aplicación.

1. Diseño de la Arquitectura:

La arquitectura de la aplicación se basa en el stack LAMP con Laravel 10. Se adoptó un enfoque MVC, con Blade, JavaScript Vanilla, AJAX, CSS y HTML en el frontend, Laravel para la lógica de negocio y MySQL para la persistencia de datos.

2. Selección de Tecnologías:

Se eligió Laravel 10 por su estructura clara y convenciones que ayudan a mantener el código organizado y coherente. Para la base de datos, se utilizó MySQL, y para las consultas, se optó por consultas raw de la clase DB de Laravel.

3. Desarrollo de Funcionalidades:

El desarrollo de funcionalidades se realizó en varias etapas, comenzando con la investigación de Laravel y el análisis de las necesidades del proyecto. A continuación, se organizó el trabajo, se creó el Anteproyecto y el logo, y se realizaron los Diagramas de Gantt, E-R, Relacional y UML. Posteriormente, se diseñó la UI/UX y se prepararon los entornos de desarrollo/producción. Después de preparar el repositorio GIT y configurar Apache, se creó la Base de Datos, seguidos por la creación de Controladores, Middlewares y Vistas. Finalmente, se estilizaron las Vistas y se añadió la Interactividad Frontend.

4. Integración de la API de Google Maps:

La API de Google Maps se integró en la lectura del crimen, permitiendo mostrar la localización del crimen en un iframe.

5. Pruebas:

Se realizaron pruebas manuales exhaustivas, verificando todas las interacciones previstas con la aplicación, utilizando diversos tipos de parámetros y probando todas las posibilidades de error previstas.

6. Seguridad:

Se implementaron medidas de seguridad para proteger contra vulnerabilidades comunes y ataques cibernéticos, como SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Session Hijacking, Inyecciones de archivos, Ataques de Directorio Transversal, Fuerza Bruta y Ataques de Diccionario. Además, se implementó un sistema de baneo y se utilizó un middleware riguroso para controlar el acceso a la aplicación.

7. Implementación y Despliegue:

La implementación y el despliegue de la aplicación se realizaron a través de un script en bash que utiliza Composer, Apache, MySQL y un repositorio Git remoto.

8. Mantenimiento y Actualizaciones:

Se planificó un mantenimiento regular y se estableció un proceso para las actualizaciones futuras, garantizando que la aplicación se mantenga actualizada y siga siendo segura y eficiente en el futuro.

Proyecto 2 - DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Fase de pruebas.

La fase de pruebas de la aplicación Crime Case Central se llevó a cabo con un enfoque meticuloso y detallado. Durante esta fase, se realizaron una serie de pruebas exhaustivas para validar cada aspecto de la aplicación.

Se inició con la verificación de todas las interacciones previstas con la aplicación. Esto incluyó la navegación por todas las secciones y módulos, la utilización de todas las herramientas y funcionalidades disponibles, y la realización de todas las operaciones posibles dentro de la plataforma.

Se utilizó una variedad de parámetros durante estas pruebas para simular diferentes escenarios y situaciones. Esto permitió evaluar cómo la aplicación se comporta bajo diferentes condiciones y con diferentes tipos de datos.

Además, se llevaron a cabo pruebas específicas para probar todas las posibilidades de error previstas. Esto implicó introducir datos incorrectos o inválidos, intentar realizar operaciones no permitidas y probar los límites de la aplicación.

Al final de esta fase de pruebas, se pudo confirmar que la aplicación Crime Case Central está lista para su implementación. La aplicación demostró ser robusta y eficiente, cumpliendo con todas las expectativas y requisitos establecidos para ella. Con la finalización exitosa de esta fase de pruebas, la aplicación está ahora un paso más cerca de revolucionar la manera en que se maneja la información relacionada con crímenes.

Formación de los usuarios finales.

Los usuarios finales serán formados bajo un plan de formación el cual se puede ver detallado en el punto 11 de este documento. Para reforzar el plan de formación realizado, los usuarios dispondrán de un Manual de Usuario, el cual contiene unos videos tutoriales que mostrarán de forma breve y concisa el uso del aplicativo.

Proyecto 2ºDAW. Autores:
Ahmad Hassan Khamis
y Christian Miguel Giménez Puerma

Los enlaces a los vídeos son los siguientes:

- **Tutorial para los usuarios lectores:** https://drive.google.com/file/d/1f-JzXopCg1UcmW4PKaPui6Ta9CLytQOu/view?usp=sharing
- **Tutorial para los usuarios editores:** https://drive.google.com/file/d/1f-JzXopCg1UcmW4PKaPui6Ta9CLytQOu/view?usp=sharing

En cuanto a los usuarios administradores y moderadores, al ser parte de nuestra organización, serán instruidos bajo el plan formativo.

Lista de modificaciones o ampliaciones futuras que se podrían aplicar a la aplicación proyectada

Para futuras versiones, se implementará el guardado de evidencias en formato video, foros de chat de investigación, lista de contactos, posibles implementaciones con sistemas de videovigilancia con sistemas métricos, una versión de la aplicación desktop y una versión móvil descargable. También se pondrá sistemas de información y avisos y notificaciones para informar de cualquier cambio o actualización en un listado de favoritos que podrá designar el usuario.

Plan de mantenimiento de la aplicación.

Mantenimiento Regular:

Se realizará un mantenimiento regular de la aplicación para garantizar su funcionamiento óptimo. Esto incluirá la monitorización constante del rendimiento de la aplicación, la identificación y corrección de cualquier problema o error que pueda surgir, y la realización de ajustes necesarios para mejorar la eficiencia y la usabilidad de la aplicación.

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Actualizaciones de Seguridad:

Se implementarán actualizaciones de seguridad de manera oportuna para proteger la aplicación contra las últimas amenazas y vulnerabilidades. Esto incluirá la aplicación de parches de seguridad, la actualización de las dependencias del software a sus últimas versiones seguras y la revisión regular de las prácticas de seguridad para garantizar que cumplen con los estándares de seguridad actuales.

Actualizaciones Funcionales:

Se realizarán actualizaciones funcionales para añadir nuevas características y mejoras a la aplicación según las necesidades y los comentarios de los usuarios. Esto puede incluir la mejora de las funcionalidades existentes, la adición de nuevas herramientas y características, y la mejora de la interfaz de usuario para mejorar la experiencia del usuario.

Pruebas de Regresión:

Después de cada actualización, se realizan pruebas de regresión para asegurar que las nuevas modificaciones no hayan introducido nuevos errores en la aplicación y que todas las funcionalidades existentes sigan funcionando correctamente.

Documentación:

Se mantendrá una documentación actualizada de la aplicación que refleje todas las actualizaciones y cambios realizados. Esto facilitará el seguimiento de las modificaciones y ayudará en la resolución de problemas y en la planificación de futuras actualizaciones.

Formación de Usuarios:

En caso de actualizaciones significativas que cambien la forma en que los usuarios interactúan con la aplicación, se proporcionará formación o guías de usuario actualizadas para ayudar a los usuarios a familiarizarse con las nuevas características o cambios.

Soporte al Usuario:

Se proporcionará soporte continuo a los usuarios para resolver cualquier problema o duda que puedan tener con respecto a la aplicación. Esto incluirá la resolución de problemas técnicos, la ayuda con el uso de la aplicación y la respuesta a las preguntas de los usuarios.

Proyecto 2 DAW Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Este plan de mantenimiento y actualizaciones garantizará que la aplicación Crime Case Central siga siendo segura, eficiente y útil para los usuarios, y que pueda adaptarse y evolucionar según las necesidades cambiantes de los usuarios y el entorno de seguridad.

6. Estudio de la viabilidad del proyecto

Análisis del Mercado:

El mercado de herramientas de gestión de información delictiva es amplio y diverso, con una creciente demanda de soluciones que mejoren la eficiencia en la investigación criminal. La necesidad de colaboración entre las fuerzas del orden y la rápida recuperación de información relevante son factores clave en la toma de decisiones de compra para este tipo de herramientas.

Justificación económica:

Suponiendo que el proyecto ha sido contratado por un año, el presupuesto propuesto de 60 € al mes resultaría en un gasto total de 720 € al año. Este gasto cubriría los costos de infraestructura, la implementación de la API de Google Maps y el aumento de la capacidad del servidor.

Los ingresos para el primer año de operación provendrían de varias fuentes, incluyendo el aporte personal de los socios, la inversión en criptomonedas, y varias subvenciones y ayudas. Estos ingresos son suficientes para cubrir el gasto total de 720 €, asumiendo que las estimaciones son correctas.

A continuación, presentamos una tabla explicativa de los ingresos y gastos anuales para justificar la viabilidad del proyecto Crime Case Central. Esta tabla proporciona una visión general de cómo los ingresos provenientes de diversas fuentes, incluyendo el aporte personal de los socios, la inversión en criptomonedas, y varias subvenciones y ayudas, pueden soportar los gastos asociados con el desarrollo y mantenimiento del proyecto.

Proyecto 2º DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Es importante destacar que los datos presentados en esta tabla son ficticios y se han proporcionado únicamente con fines ilustrativos. Estamos a la espera de la respuesta de las entidades correspondientes para obtener los datos reales de las subvenciones y ayudas. Una vez que tengamos esta información, podremos ajustar las cifras para reflejar la situación financiera real del proyecto.

Concepto	Cantidad (€)
Aporte personal de socios	5000
Inversión en criptomonedas	3000
Subvención pública del SEPE	2000
Ayudas ICO	1500
Ayudas de las Comunidades Autónomas	1000
Total de Ingresos	**12500**
Costo de infraestructura (VPS)	30 x 12 = 360
Implementación de la API de Google Maps	20 x 12 = 240
Aumento de la capacidad del servidor	10 x 12 = 120
Total de Gastos	**720**

Análisis de Viabilidad Técnica:

Crime Case Central se basa en tecnologías web probadas y ampliamente utilizadas, lo que garantiza su compatibilidad y accesibilidad en una variedad de dispositivos y plataformas. Esto significa que la aplicación puede ser utilizada en cualquier lugar y en cualquier momento, siempre que el usuario tenga acceso a Internet.

Además, su diseño modular y escalable permite una fácil adaptación a las necesidades específicas de cada usuario. Esto significa que la aplicación puede ser personalizada para satisfacer las necesidades individuales de cada usuario, lo que aumenta su utilidad y eficacia.

La aplicación también está diseñada para incorporar nuevas funcionalidades en el futuro. Esto asegura que la aplicación puede evolucionar y adaptarse a las cambiantes necesidades y expectativas de los usuarios y del campo de la criminología en general.

En resumen, la viabilidad técnica de Crime Case Central está asegurada por su uso de tecnologías web probadas, su diseño modular y escalable, y su capacidad para

adaptarse y evolucionar con el tiempo. Todo esto, junto con su enfoque en la seguridad y la protección de los datos, hace que Crime Case Central sea una herramienta valiosa y viable para los profesionales de la ley y la criminología.

Análisis de Viabilidad Económica:
La implementación de Crime Case Central conlleva costos asociados al desarrollo del software, la infraestructura de alojamiento y el soporte técnico. Sin embargo, estos costos se ven compensados por los beneficios en términos de eficiencia operativa y mejora en la resolución de casos criminales. Se espera que el retorno de la inversión sea positivo a corto y largo plazo.

Análisis de Viabilidad Legal y Regulatoria:
Crime Case Central debe cumplir con las regulaciones y normativas de protección de datos y privacidad aplicables en las jurisdicciones donde se utilice. Es fundamental garantizar la seguridad y confidencialidad de la información delictiva gestionada por la aplicación, así como el cumplimiento de los protocolos de cadena de custodia y preservación de evidencia.

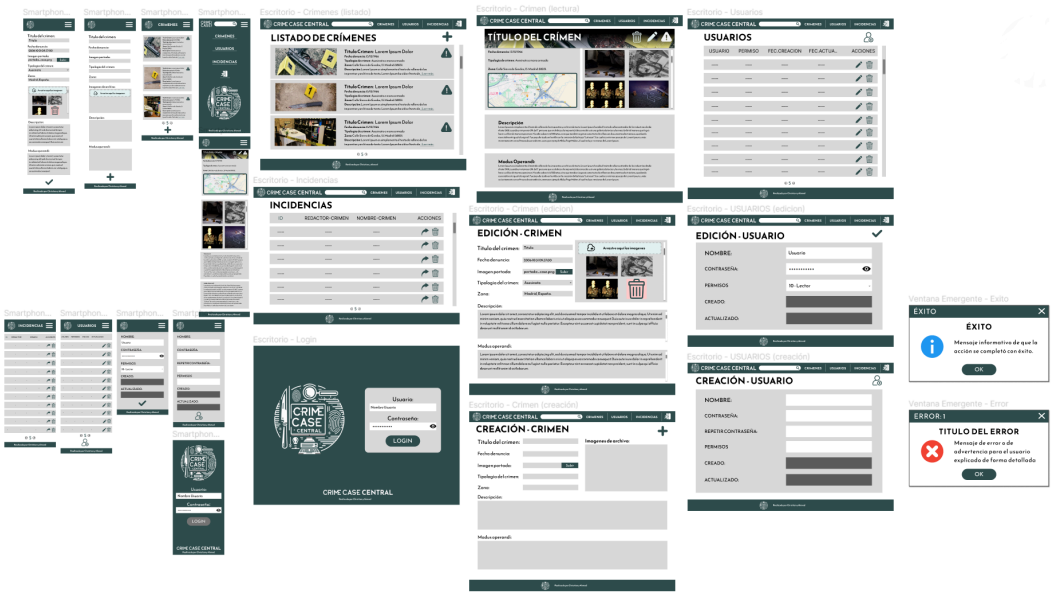
Por ello, Crime Case Central cumple con la normativa legal de la Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales. Crime Case Central y todos sus servicios están estructurados de manera ecológica con energías renovables como o tales como paneles solares, cumpliendo así la Ley 26/2007, de Responsabilidad Medioambiental en España.

Proyecto 2º DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

7. Documentación del diseño e implementación de la solución adoptada.

Prototipo de la aplicación. Diseño de interfaces.

El prototipo de la aplicación es el siguiente, el cual está detallado en la guía de estilos para su correcta implementación.



Como se puede observar, el aplicativo contará con versión móvil y escritorio, ofreciendo en todo momento interfaces que brindan al usuario una accesibilidad adecuada, evitando sobrecargar la página de elementos excesivos, aportando una UI/UX minimalista e intuitiva que cumple con las funcionalidades descritas.

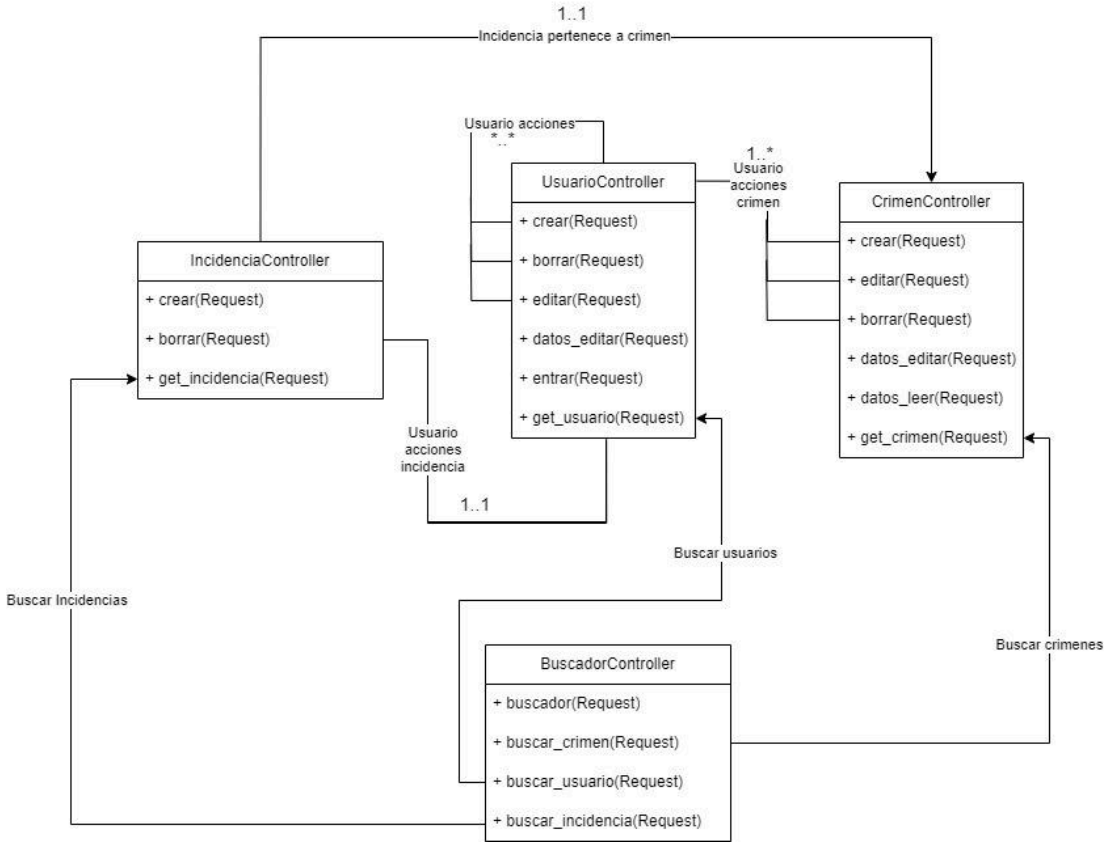
Diseño lógico de la aplicación a desarrollar.

Diagrama UML

Debida a la arquitectura MVC de Laravel, y a las necesidades de este proyecto, el sistema de controladores, los cuales realizan toda la lógica de negocio, reciben los

datos de las vistas y a su vez, interactúan con la base de datos, tienen la siguientes relaciones las cuales hemos representado en el siguiente diagrama UML:

- Un usuario crea/edita/borra un crimen y a su vez un usuario puede crear/editar/borrar a otros usuarios.
- Una incidencia pertenece a un crimen y es creada/borrada por un usuario.
- El buscador realiza consultas para obtener un listado de incidencias, usuarios y/o crímenes.



Descripción modular del software a desarrollar:

Los módulos funcionales del proyecto, son los descritos a continuación:

Módulo funcional de la aplicación	Descripción	Contenido a trabajar
Módulo de vista	El módulo de vista proporcionará a los usuarios una interfaz	Se crearán las distintas plantillas Blade que proporcionarán, de forma

	intuitiva y eficiente para acceder y visualizar la información relacionada con los crímenes registrados en el sistema de gestión. Este módulo ofrecerá una variedad de características diseñadas para facilitar la exploración y comprensión de los datos.	modulada, las distintas vistas, acordes al diseño adjunto.
Módulo de base de datos	El módulo de base de datos constituirá la piedra angular para la gestión del modelo lógico de persistencia de datos de la aplicación de gestión de crímenes. Estará diseñado para ofrecer una estructura robusta y eficiente que permita la manipulación y almacenamiento de la información de manera coherente y segura.	Creación de la base de datos, así como de la tabla necesaria acorde al esquema Entidad-Relación adjunto. Uso de la interfaz de Laravel "DB" que realizará en cada uno de los controladores todo el sistema CRUD necesario en la aplicación.
Módulo archivador	El módulo archivador desempeña un papel crucial en la gestión de las imágenes de archivo asociadas a los crímenes registrados en la aplicación. Está diseñado para proporcionar un sistema de	Creación de una ruta en el servidor, la cual será accesible a través de Laravel, de forma que la aplicación gestione los ficheros que lleguen al servidor web, almacenando en una serie de rutas sólo accesibles

Proyecto 2º DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

	almacenamiento organizado y seguro para estas imágenes, permitiendo un acceso rápido y eficiente cuando sea necesario. Además contará con un sistema Drag & Drop para que el usuario pueda subir o eliminar las imágenes del archivador de manera cómoda e intuitiva.	desde el propio servidor, guardando las rutas en su correspondiente tabla de la BBDD. Creación del módulo de JavaScript Drag & Drop, permitiendo la subida y borrado de imágenes a la interfaz web de la aplicación.
Módulo de creación de crímenes	El módulo de creación de crímenes es una parte esencial de la aplicación de gestión de crímenes, diseñado para facilitar la entrada de información sobre incidentes delictivos en el sistema. Este módulo proporciona una interfaz intuitiva y eficiente para que los usuarios registren nuevos crímenes de manera precisa y completa.	Crear el controlador asociado, el cual, recibiendo los datos desde el frontend, realizará la conexión y lógica de negocio pertinente, sirviendo de puente entre las vistas y el módulo de base de datos de la aplicación.
Módulo listado de crímenes	El módulo de listado de crímenes estará diseñado para proporcionar a los usuarios una vista organizada y accesible de todos los crímenes registrados en el sistema. Este módulo permite la	Realización de la parte lógica que permita mostrar al usuario, de forma intuitiva y dinámica, un listado recibido desde la base de datos. Creación de la vista pertinente. Creación de

Proyecto 2º DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

	consulta y exploración de los datos de manera eficiente, facilitando la identificación y el análisis de los crímenes según diversos criterios, así como pudiendo ordenar los datos por cada una de las columnas, contando con un sistema de paginación intuitivo.	las rutas y del método en el controlador pertinente y realización de la ordenación de las columnas cuando el usuario haga clic en ellas con JavaScript, respetando la paginación.
Módulo de edición de crímenes	El módulo de listado de crímenes proporcionará a los usuarios una vista organizada y accesible de todos los crímenes registrados en el sistema. Este módulo permite la consulta y exploración de los datos de manera eficiente, facilitando la edición de todos y cada uno de los datos del crímen.	Creación de los formularios necesarios para la edición. Recuperación de los datos detallados para brindar su posterior edición al usuario. Realización de la lógica asociada a la funcionalidad pedida.
Módulo de lectura de crímen	El módulo de lectura de crímenes brindará a los usuarios una interfaz intuitiva y detallada para explorar la información de los crímenes registrados en el sistema. Este módulo permitirá a los usuarios visualizar de manera eficiente los	Recuperación de los datos de la base de datos, mostrando de manera clara y concisa todos los datos que se almacenarán previamente en la creación del crímen. Crear las funcionalidades de botón de borrado así como de edición del

Proyecto 2º DAW. Autores:
Ahmed Hasan Khamis
y Christian Miguel Giménez Puerma

	detalles de los crímenes, así como realizar acciones como la edición (llevando al usuario al módulo pertinente) y la eliminación de crímenes.	crimen.
Módulo listado de usuarios	El módulo de listado de usuarios ofrece una interfaz minimalista, en la cual se muestra el listado de usuarios a los administradores, teniendo de forma intuitiva la opción de crear un nuevo usuario así como de editar o borrar uno ya existente. También se podrá ordenar los datos por cada una de las columnas, contando con un sistema de paginación intuitivo.	Creación de la interfaz, listado de los usuarios recogiendo los datos de la base de datos, unión de la interfaz con las acciones y módulos pertinentes, ordenación y paginación del listado.
Módulo de creación de usuarios	El módulo de creación de usuarios brinda un formulario sencillo en el cual los administradores podrán dar de alta a los nuevos usuarios, obligando a insertar una password segura (debido a el riesgo tan alto de los datos de este aplicativo, las contraseñas deben ser	Crear el controlador asociado, el cual, recibiendo los datos desde el frontend, realizará la conexión y lógica de negocio pertinente, sirviendo de puente entre las vistas y el módulo de base de datos de la aplicación. Realizar la validación por

Proyecto 2º DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

	seguras y generadas por los administradores) y la gestión de permisos, la cual restringirá al usuario el uso del aplicativo en función de los mismos.	JavaScript del cumplimiento de los datos, verificando con expresiones regulares, para así asegurar el correcto envío, además se realizará la misma verificación en el servidor.
Módulo de edición de usuarios	El módulo de edición de usuarios aportará una interfaz idéntica al de creación, pero recuperando los datos de la base de datos, asegurándose de traer datos sensibles como la contraseña de manera encriptada.	Creación de los formularios necesarios para la edición. Recuperación de los datos detallados para brindar su posterior edición al administrador. Realización de la lógica asociada a la funcionalidad pedida.
Módulo listado de incidencias	El módulo de listado de incidencias aportará la sencillez de poder acceder con un sólo clic a la lectura del crimen del cual se dió de alta la incidencia, para así tratar de resolverla. También se podrá ordenar el listado y borrar la incidencia en cuestión	Creación de la interfaz, listado de las incidencias, recogiendo los datos de la base de datos, unión de la interfaz con las acciones y módulos pertinentes, ordenación y paginación del listado.
Módulo de búsqueda de datos	El módulo de búsqueda de datos permitirá al usuario hacer una búsqueda amplia dentro	Creación del controlador pertinente, el cual, dependiendo de la ruta por la cual le venga la

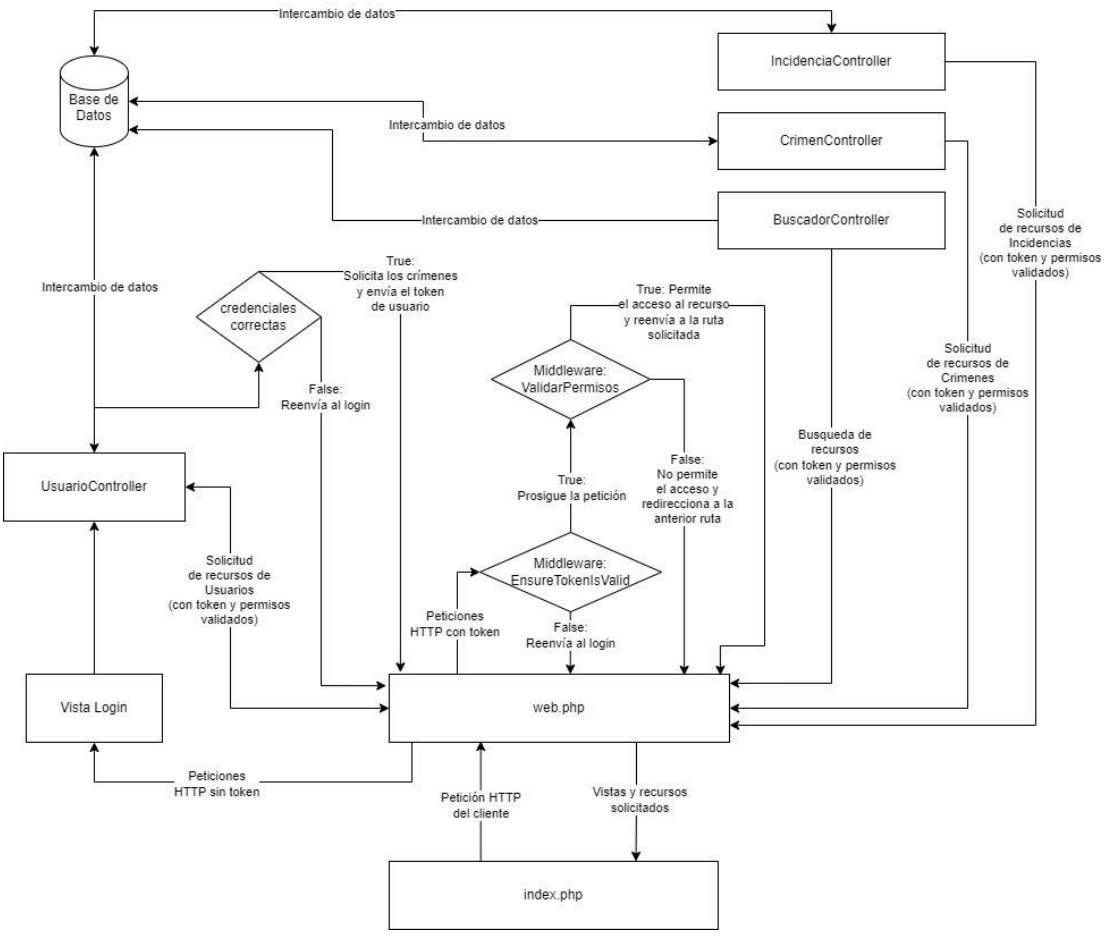
	de los registros de los crímenes, independientemente del campo a buscar, el buscador filtrará los crímenes según el término buscado. Además brindará a los administradores, en las secciones pertinentes, buscar también incidencias y usuarios.	petición de búsqueda, realizará la consulta a la base de datos pertinente, mostrando al usuario un listado de los resultados que coinciden con su búsqueda.
Módulo de control de permisos	El módulo de control de permisos asegura que cada usuario acceda solo a las funciones y datos que su rol permite. Mediante la asignación de permisos específicos, se facilita una gestión eficaz y segura de las capacidades de cada usuario dentro de la aplicación, garantizando que la modificación de datos y gestión de usuarios sólo esté disponible para aquellos con la autoridad necesaria.	Creación del middleware pertinente, el cual dependiendo de los permisos que tenga cada usuario, permitirá o restringirá el acceso a determinados módulos, así como ocultará las vistas y los iconos de las secciones no autorizadas.
Módulo de control de acceso y de sesión	El módulo de control de acceso y de sesión protege la aplicación contra accesos no	Creación del middleware pertinente, el cual, junto al controlador de usuario, creará el token volátil y

Proyecto 2º DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

	autorizados y gestiona las sesiones de los usuarios de manera segura. Con un sistema de autenticación robusto y un control estricto de sesiones, se previene el ingreso indebido y se mantiene la integridad de la sesión del usuario, asegurando que la información crítica se maneje de forma confidencial y segura, penalizando con hasta 2 horas sin poder volver a iniciar sesión a los usuarios que realicen más de 10 intentos fallidos de sesión	único de sesión, validará las credenciales del token de acceso, así como contabilizará en la base de datos en nº de intentos fallidos, los cuales si superan los 10 intentos, acarreará un baneo de 2 horas al usuario, guardando todo ello en la base de datos para prevenir el acceso de ese usuario a través de otros medios (otro navegador, dispositivo, VPN, etc.)

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Diagrama de flujo del aplicativo:



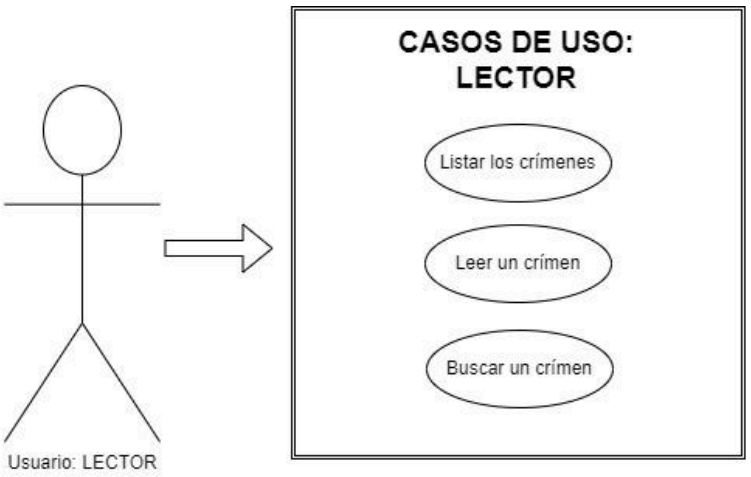
Todo el flujo del programa pasa por el enrutador **web.php**, el cual, apoyándose en un Middleware de gestión de token (*EnsureTokenIsValid*) y en un Middleware de gestión de permisos (*ValidarPermisos*), utiliza el resto de controladores para enrutar las peticiones del usuario, pidiendo acciones a los controladores, los cuales consultarán/modificarán la base de datos y generarán las vistas solicitadas, todas ellas renderizando en index.php.

Diagrama casos de uso:

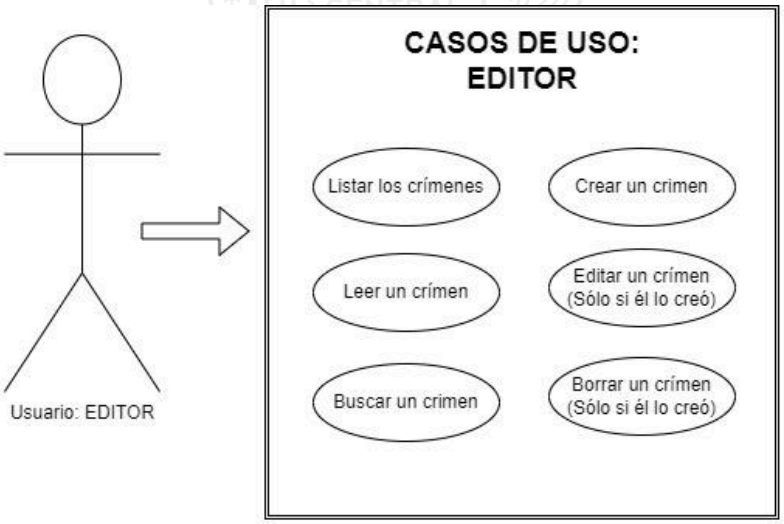
El aplicativo se basa en los siguientes casos de uso, extendiendo su funcionalidad principal de creación de crímenes, con todo lo que ello conlleva (sistema CRUD con subida y almacenaje de ficheros confidenciales, etc.), a la implementación de un sistema de creación de incidencias, el cual permitirá reportar errores, fallos o desinformación en los crímenes, volviendo la gestión de los mismos más robusta y acorde a las necesidades de este proyecto, a la implementación de una gestión de distintos tipos de usuario, restringiendo el uso del aplicativo dependiendo del rol

asignado y a la implementación de un sistema de búsqueda, el cual permitirá hacer un filtrado de los datos requeridos por el usuario.

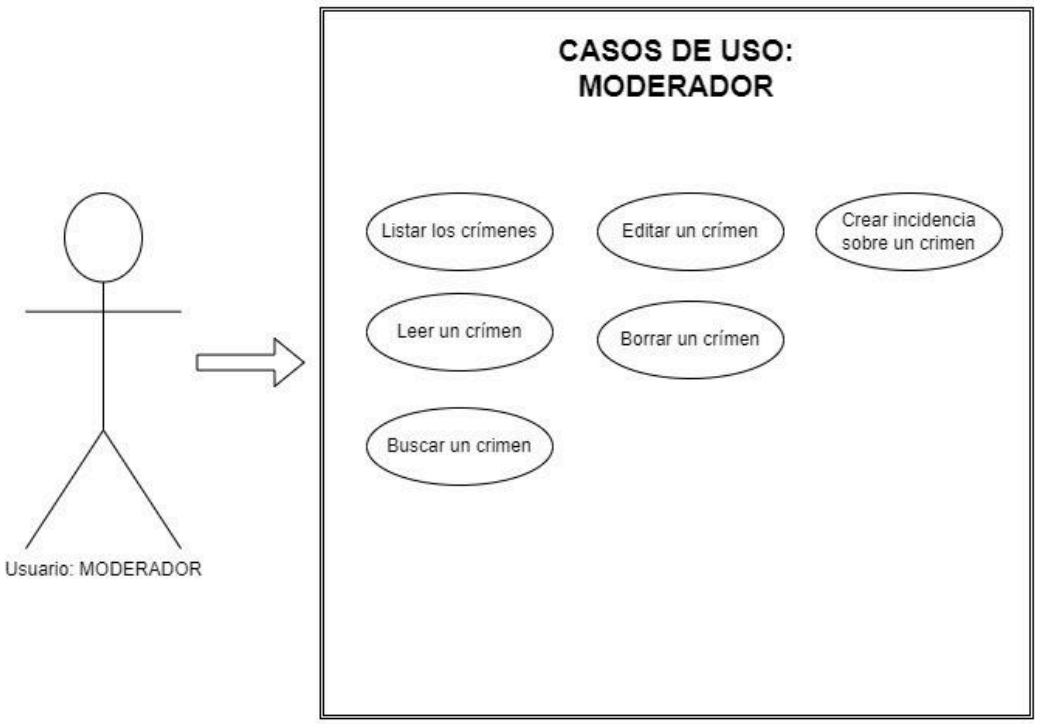
Caso de uso Usuario tipo Lector



Caso de uso Usuario tipo Editor

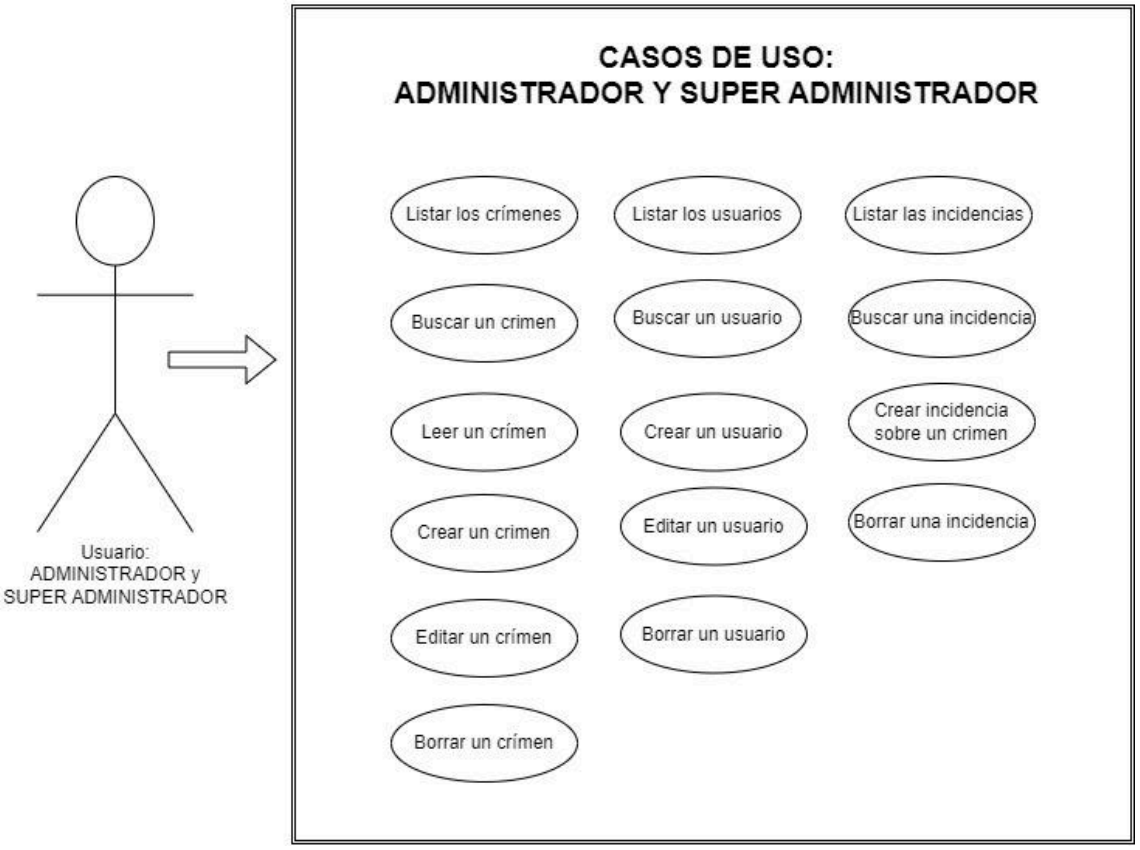


Caso de uso Usuario tipo Moderador



Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Caso de uso Usuario tipo Administrador/Super Administrador



Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Diseño de la base de datos:

Diagrama Entidad-Relación

El siguiente diagrama Entidad-Relación, representa la persistencia de datos que el aplicativo Crime Case Central requiere, el cual consta principalmente del registro de crímenes, asociados al usuario creador, y a su vez, el alta de incidencias hecha por el usuario y su pertenencia al crimen en cuestión.

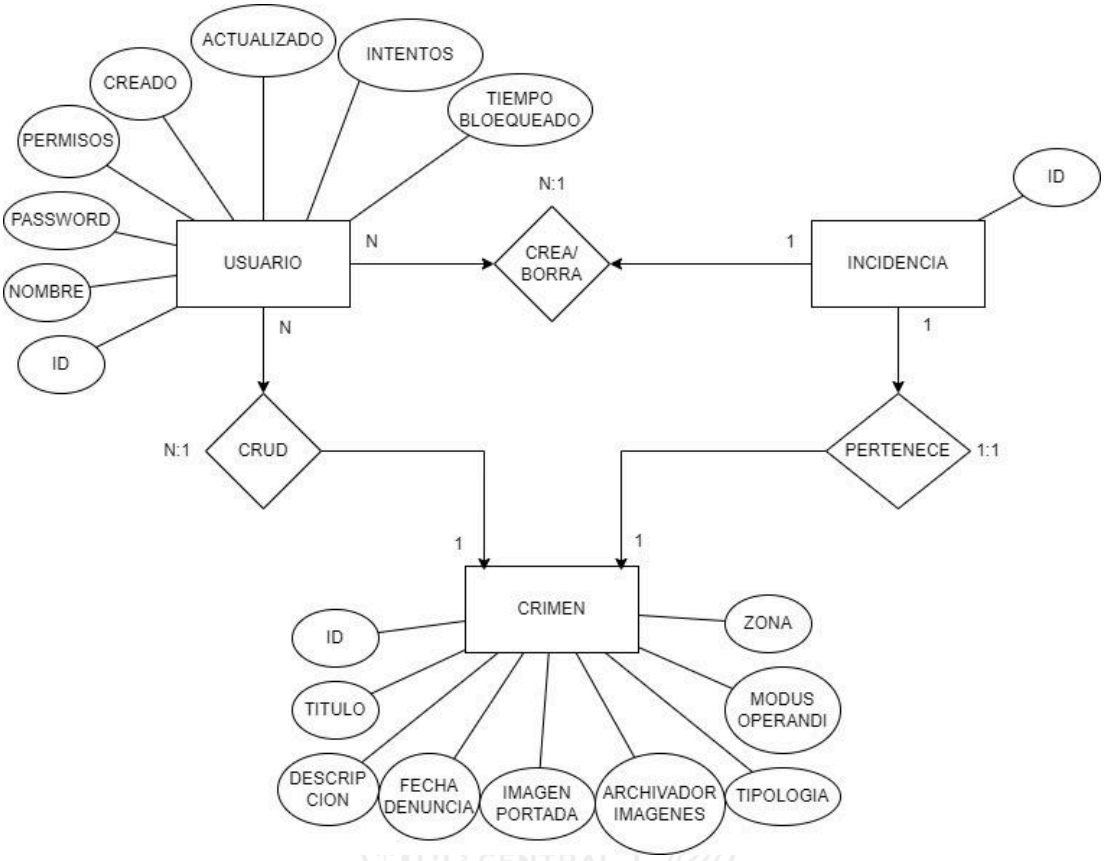
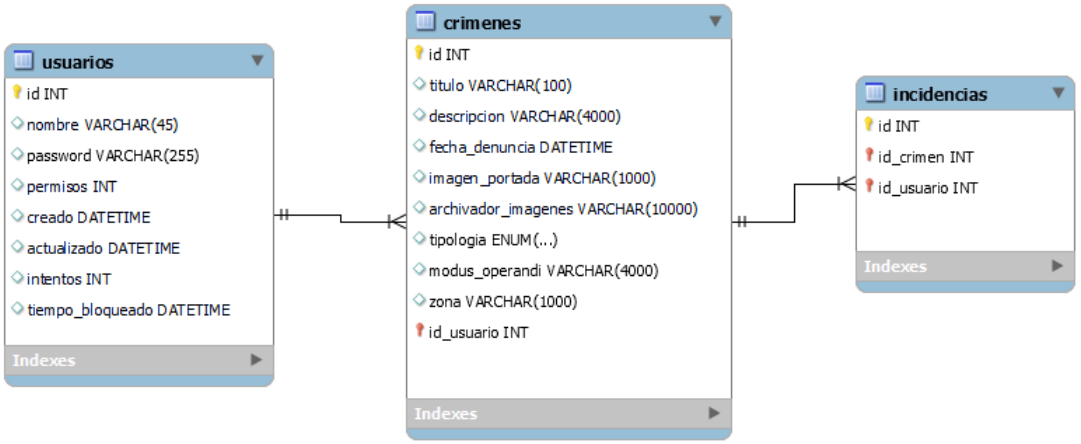


Diagrama Relacional

El diagrama E-R anterior se traduce en el siguiente diagrama Relacional:



Justificación del no uso del ORM Eloquent de Laravel

Justificación del Uso de Sentencias RAW en Laravel para Consultas Simples

En el desarrollo de nuestra aplicación web, nos enfrentamos a la decisión de cómo manejar consultas simples a la base de datos. Si bien Laravel ofrece un poderoso

ORM (Mapeo Objeto-Relacional) que facilita la interacción con la base de datos de una manera intuitiva y eficiente, en algunos casos, utilizarlo puede resultar excesivo y poco eficiente.

Las sentencias RAW, que permiten ejecutar consultas SQL directamente en Laravel, ofrecen una alternativa más adecuada para estas situaciones. A continuación, se presentan algunas razones que justifican el uso de sentencias RAW en lugar de ORM para consultas simples:

Simplicidad y Claridad: Las consultas RAW son simples y directas. Para consultas simples que no requieren la manipulación compleja de modelos y relaciones, escribir consultas SQL directamente puede ser más claro y fácil de entender para otros desarrolladores que revisen el código en el futuro.

Eficiencia: Al evitar la sobrecarga de utilizar un ORM para consultas simples, podemos lograr un mejor rendimiento al ejecutar consultas directas a la base de datos. Esto es especialmente importante en consultas que se ejecutan con frecuencia o en entornos de alta concurrencia.

Flexibilidad: Las sentencias RAW ofrecen una mayor flexibilidad en la construcción de consultas. Podemos aprovechar todas las características del lenguaje SQL y realizar consultas más avanzadas que pueden no ser posibles o ser más complicadas de lograr utilizando el ORM de Laravel.

Proyecto 2º DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Otras estructuras de datos que se utilizan en la aplicación.

Estructuras de sesión de Usuarios: Se utilizan para almacenar información sobre los usuarios autorizados que acceden a la aplicación, incluyendo detalles de autenticación, roles y permisos asignados.

Estructuras de Evidencia (Archivador de imágenes e imágenes de portada): Estas estructuras almacenan imágenes relacionadas con casos criminales individuales, recolectando las pruebas para mostrar la evidencia asociada.

Estructuras de Geo-Localización: Estas estructuras almacenan información geoespacial relacionada con los crímenes, como ubicaciones de incidentes, áreas de patrullaje y puntos de interés relevantes. Las cuales son almacenadas en el campo "zona" y son mostradas al usuario a través de la API de Google Maps.

Estas estructuras de datos adicionales son fundamentales para la funcionalidad y eficacia de Crime Case Central, permitiendo una gestión integral de la información delictiva y facilitando la colaboración y el análisis en la investigación criminal.

Estudio de la seguridad de la aplicación.

Como ya se ha comentado anteriormente en el apartado "Análisis de los problemas de las aplicaciones actuales" la aplicación consta de un sólido sistema de ciberseguridad, el cual tendrá en cuenta:

Análisis de Amenazas:
Se identifican diversas amenazas potenciales a la seguridad de Crime Case Central, incluyendo accesos no autorizados, ataques de denegación de servicio, manipulación de datos y robo de información sensible. Estas amenazas podrían comprometer la eficacia de la investigación criminal y poner en riesgo la confianza en la aplicación.

Proyecto 2º DAW. Autores:
Ahmed Hassan Khamis
y Cristian Miguel Giménez Puerma

Medidas de Seguridad Implementadas:
Crime Case Central cuenta con múltiples medidas de seguridad para mitigar las amenazas identificadas. Estas incluyen:

Autenticación y Autorización: Se requiere autenticación de usuarios mediante credenciales seguras, como contraseñas robustas o autenticación de dos factores. Además, se aplican controles de autorización para limitar el acceso a funciones y datos según el rol y los privilegios del usuario.

Encriptación de Datos: Todos los datos almacenados y transmitidos por Crime Case Central se cifran utilizando algoritmos de encriptación robustos, garantizando que la información sensible está protegida contra accesos no autorizados.

Auditoría de Acceso: Se registran y supervisan todas las actividades realizadas dentro de la aplicación, lo que permite detectar y responder a posibles intrusiones o comportamientos sospechosos.

Protección contra Ataques: Se implementan medidas de seguridad para proteger la aplicación contra ataques de denegación de servicio, inyección de código y otros vectores de ataque comunes, utilizando firewalls, filtros de entrada y otras herramientas de seguridad perimetral.

Evaluación Continua:
La seguridad de Crime Case Central se somete a evaluaciones regulares y pruebas de penetración para identificar posibles vulnerabilidades y mejorar constantemente las medidas de seguridad. Además, se mantiene actualizado el software y se aplican parches de seguridad de manera proactiva para mitigar riesgos potenciales.

8. Código fuente documentado

Como se indicó en el apartado de la solución propuesta, se adjunta aquí el código asociado al aplicativo, mostrando el uso de todos los módulos, controladores y middleware citados.

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Código Enrutador

Fichero enrutamiento web.php

```php
<?php

use App\Http\Middleware\ValidarPermisos;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\UsuarioController;
use App\Http\Controllers\CrimenController;
use App\Http\Controllers\IncidenciaController;
use App\Http\Controllers\BuscadorController;
use App\Http\Middleware\EnsureTokenIsValid;

/*
 Fichero principal de sistema de enrrutamiento, en el se encuentran todas las rutas definidas
 así como los controladores, vistas y middleware asociadas.
 */

//Ruta principal (login)
Route::get('/', function () {
    return view('login');
})->name("login");

//Listado de crimenes, es lo 1º que verá el usuario
Route::get('/listado-crimen', function () {
    return view('listado-crimen');
})->name("listado-crimen")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Listado de usuarios
Route::get('/listado-usuario', function () {
    return view('listado-usuario');
})->name("listado-usuario")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Listado de incidencias
Route::get('/listado-incidencia', function () {
    return view('listado-incidencia');
})->name("listado-incidencia")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Lectura de un crimen
Route::get("/leer-crimen", function() {
    return view("leer-crimen");
})->name("leer-crimen")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Creación de un crimen
Route::get("/crear-crimen", [CrimenController::class, "get_tipologias"])->name("crear-crimen")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Edición de un crimen
Route::get("/editar-crimen", function() {
    return view("editar-crimen");
})->name("editar-crimen")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Creación de un usuario
Route::get("/crear-usuario", function() {
    return view("crear-usuario");
})->name("crear-usuario")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Edición de un usuario
Route::get("/editar-usuario", function() {
    return view("editar-usuario");
})->name("editar-usuario")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Lleva al controlador que carga los datos del usuario seleccionado antes de editar
Route::get('/editar-usuario-cargadb', [UsuarioController::class, "datos_editar"])->name("editar-usuario-cargadb")->middleware(ValidarPermisos::class);

//Lleva al controlador que valida el login
Route::post('/validar-login', [UsuarioController::class, "entrar"])->name("validar-login");

//Lleva al controlador que crea el usuario
Route::post('/crear-usuario-db', [UsuarioController::class, "crear"])->name("crear-usuario-db")->middleware(ValidarPermisos::class);

//Lleva al controlador que edita el usuario
Route::post('/editar-usuario-db', [UsuarioController::class, "editar"])->name("editar-usuario-db")->middleware(ValidarPermisos::class);

//Lleva al controlador que borra el usuario
Route::get('/borrar-usuario', [UsuarioController::class, "borrar"])->name("borrar-usuario")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Trae la lista de usuarios
Route::get('/get-usuario', [UsuarioController::class, "get_usuario"])->name("get-usuario")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Trae la lista de incidencias
Route::get('/get-incidencia', [IncidenciaController::class, "get_incidencia"])->name("get-incidencia")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Trae la lista de crimenes
Route::get('/get-crimen', [CrimenController::class, "get_crimen"])->name("get-crimen")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Lleva al controlador que borra el usuario
Route::get('/borrar-incidencia', [IncidenciaController::class, "borrar"])->name("borrar-incidencia")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Lleva al controlador que crea la incidencia
Route::get('/crear-incidencia-db', [IncidenciaController::class, "crear"])->name("crear-incidencia-db")->middleware(ValidarPermisos::class);

//Lleva al controlador que crea el usuario
Route::post('/crear-crimen-db', [CrimenController::class, "crear"])->name("crear-crimen-db")->middleware(ValidarPermisos::class);

//Lleva al controlador que extraer los datos
Route::get('/editar-crimen-cargadb', [CrimenController::class, "datos_editar"])->name("editar-crimen-cargadb")->middleware(ValidarPermisos::class);

//Lleva al controlador que edita el crimen
Route::post('/editar-crimen-db', [CrimenController::class, "editar"])->name("editar-crimen-db")->middleware(ValidarPermisos::class);

//Lleva al controlador que extraer los datos
Route::get('/leer-crimen-db', [CrimenController::class, "datos_leer"])->name("leer-crimen-db")->middleware(ValidarPermisos::class);
//Lleva al controlador que borra el crimen
Route::get('/borrar-crimen', [CrimenController::class, "borrar"])->name("borrar-crimen")->middleware(EnsureTokenIsValid::class)->middleware(ValidarPermisos::class);

//Buscador por POST
Route::post('/buscador', [BuscadorController::class, "buscador"])->name("buscador")->middleware(ValidarPermisos::class);

//Buscador por GET
Route::get('/buscador', [BuscadorController::class, "buscador"])->name("buscador")->middleware(ValidarPermisos::class);

//Buscador de incidencias
Route::get('/buscar-incidencia', [BuscadorController::class, "buscar_incidencia"])->name("buscar-incidencia")->middleware(ValidarPermisos::class);

//Buscador de crimenes
Route::get('/buscar-crimen', [BuscadorController::class, "buscar_crimen"])->name("buscar-crimen")->middleware(ValidarPermisos::class);

//Buscador de usuarios
Route::get('/buscar-usuario', [BuscadorController::class, "buscar_usuario"])->name("buscar-usuario")->middleware(ValidarPermisos::class);

codesnap.dev
```

Código Middlewares

Fichero middleware EnsureTokenIsValid.php

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;


class EnsureTokenIsValid
{
    /**
     * Trata una request entrante y valida si el token de sesión es válido o no.
     * Si lo es, prosigue la petición, si no lo es, redirecciona al login
     *
     * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response)  $next
     */
    public function handle(Request $request, Closure $next): Response
    {
        $tokenSession = session("token");
        $tokenGet = $request->get("winterhorn");

        //Si el token por GET no es el mismo que el de Sesión, vuelve al login, si no, prosigue
        if ((!$tokenGet || !$tokenSession) || ($tokenGet != $tokenSession) || ($tokenGet == "" || $tokenSession == "") ) {
            return redirect() -> route("login");
        }
        return $next($request);
    }
}

```

codesnap.dev

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Fichero middleware ValidarPermisos.php

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;
use Illuminate\Support\Facades\DB;

class ValidarPermisos
{
    /**
     * Intercepta cada request para validar los permisos del usuario, si no tiene permisos
     * para acceder al recurso, cierra sesión y vuelve al login.
     *
     * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response)  $next
     */
    public function handle(Request $request, Closure $next): Response
    {
        //PERMISOS:
        //10: Lectores
        //20: Editores
        //30: Moderadores
        //40: Administradores
        //50: Super Administradores

        $idUsuario = session("usuario");
        $ruta = parse_url(url()->current(), PHP_URL_PATH);

        $usuario = DB::table('usuarios')->select("permisos")->where("id", $idUsuario)->first();

        switch ($ruta) {
            //Acceso administrativo de usuarios e incidencias (sólo administradores)
            case '/listado-incidencia':
            case '/listado-usuario':
            case '/get-incidencia':
            case '/get-usuario':
            case '/editar-usuario-cargadb':
            case '/crear-usuario-db':
            case '/crear-usuario':
            case '/editar-usuario-db':
            case '/editar-usuario':
            case '/borrar-usuario':
            case '/borrar-incidencia':
            case '/buscar-incidencia':
            case '/buscar-usuario':
                switch ($usuario->permisos) {
                    case '40':
                    case '50':
                        return $next($request);
                    default:
                        return back();
                }
            //Acceso de creación de incidencias (sólo moderadores y administradores)
            case '/crear-incidencia-db':
            case '/crear-incidencia':
                switch ($usuario->permisos) {
                    case '30':
                    case '40':
                    case '50':
                        return $next($request);
                    default:
                        return back();
                }
            //Acceso de creación/edición/borrado de crímenes (sólo editores, moderadores y administradores)
            case "/crear-crimen":
            case '/crear-crimen-db':
            case '/borrar-crimen':
            case "/editar-crimen":
            case "/editar-crimen-cargadb":
            case '/editar-crimen-db':
                switch ($usuario->permisos) {
                    case "30":
                        //Los moderadores pueden borrar y editar cualquier crimen
                        if ($ruta == '/borrar-crimen' || $ruta == "/editar-crimen" || $ruta == "/editar-crimen-cargadb" || $ruta == '/editar-crimen-db') {
                            return $next($request);
                        } else {
                            return back();
                        }
                    //Los editores sólo pueden borrar o editar los crímenes que fueron creados por ellos, no los de otros usuarios
                    case '20':
                        if ($ruta == '/borrar-crimen' || $ruta == "/editar-crimen" || $ruta == "/editar-crimen-cargadb" || $ruta == '/editar-crimen-db') {
                            $check_id = DB::table('crimenes')->select("id_usuario")->where("id", $request->input("id_crimen"))->first();
                            if ($idUsuario == $check_id->id_usuario) {
                                return $next($request);
                            } else {
                                return back();
                            }
                        }

                        return $next($request);
                    case '40':
                    case '50':
                        return $next($request);
                    default:
                        return back();
                }
            //Acceso general de lectura/búsqueda/listado de crímenes (todos los usuarios loggeados)
            case '/listado-crimen':
            case '/get-crimen':
            case "/leer-crimen":
            case '/leer-crimen-db':
            case '/buscar-crimen':
            case '/buscador':
            case '/validar-login':
                switch ($usuario->permisos) {
                    case '10':
                    case '20':
                    case '30':
                    case '40':
                    case '50':
                        return $next($request);
                    default:
                        return back();
                }
            default:
                return redirect()->route("login");
        }

    }
}

```

codesnap.dev

Código Controladores

Fichero CrimenController.php

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Routing\Controller;
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
use Illuminate\Foundation\Validation\ValidatesRequests;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Str;
use Carbon\Carbon;
use \DateTimeZone;

/**
 * Controlador principal para gestionar los crímenes
 */
class CrimenController extends Controller
{
    use AuthorizesRequests, ValidatesRequests;

```

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Método crear()

```php
    ///Método para crear un crimen con los datos recibidos por /crear-crimen
    public function crear(Request $request)
    {
        //Se obtienen y filtran los datos del formulario y de sesión
        $tituloCrimen = trim($request->input("tituloCrimen"));
        $descripcion = trim($request->input("descripcion"));
        $fechaDenuncia = $request->input("fechaDenuncia");
        $tipologia = trim($request->input("tipologia"));
        $modusOperandi = trim($request->input("modusOperandi"));
        $zona = trim($request->input("zona"));
        $idUsuario = session("usuario");

        //Se inicializan para evitar errores
        $rutaImagenPortada = "";
        $rutaArchivadorImagenes = "";
        $resultado = "";

        //Comprobamos si el crimen ya existia
        $crimenExiste = DB::table("crimenes")->where("titulo", $tituloCrimen)->exists();
        $type = DB::selectOne("SHOW COLUMNS FROM crimenes WHERE Field = 'tipologia' ")->Type;
        preg_match("/^enum\(\'(.*)\'\)$/", $type, $matches);
        $tipologiasPermitidas = explode("','", $matches[1]);

        //Validamos los datos y almacenamos el error/exito de la validación
        if ($crimenExiste) {
            $titulo = "ERROR: EL CRIMEN YA EXISTE";
            $resultado = "Por favor, introduzca un crimen que no exista en la base de datos. ";
            $codigo = 1;
        } else if (strlen($tituloCrimen) <= 0 || strlen($tituloCrimen) > 100) {
            $titulo = "ERROR: TITULO DE CRIMEN NO VALIDO";
            $resultado = "Por favor, introduzca un titulo de crimen cuyos caracteres estén en el rango indicado: [1 - 100]. ";
            $codigo = 2;
        } else if (strlen($descripcion) <= 0 || strlen($descripcion) > 4000) {
            $titulo = "ERROR: DESCRIPCIÓN NO VALIDA";
            $resultado = "Por favor, introduzca una descripción del crimen cuyos caracteres estén en el rango indicado: [1 - 4000]. ";
            $codigo = 3;
        } else if (!$fechaDenuncia) {
            $titulo = "ERROR: FECHA DENUNCIA NO VALIDA";
            $resultado = "Por favor, introduzca una fecha del crimen válida con el formato indicado [DD-MM-AAAA]. ";
            $codigo = 4;
        } else if (!in_array($tipologia, $tipologiasPermitidas)) {
            $titulo = "ERROR: TIPOLOGÍA DE CRIMEN NO VALIDA";
            $resultado = "Por favor, introduzca una tipología del crimen válida. ";
            $codigo = 7;
        } else if (strlen($modusOperandi) <= 0 || strlen($modusOperandi) > 4000) {
            $titulo = "ERROR: TIPOLOGÍA DE CRIMEN NO VALIDA";
            $resultado = "Por favor, introduzca un modus operandio cuyos caracteres estén en el rango indicado: [1 - 100]. ";
            $codigo = 8;
        } else if (strlen($zona) <= 0 || strlen($zona) > 1000) {
            $titulo = "ERROR: ZONA DE CRIMEN NO VALIDA";
            $resultado = "Por favor, introduzca una zona de crimen válida. ";
            $codigo = 9;
        } else {
            //Si no hubo errores, insertamos los datos en la DB
            $resultado = DB::table("crimenes")->insert([
                "titulo" => $tituloCrimen,
                "descripcion" => $descripcion,
                "fecha_denuncia" => $fechaDenuncia,
                "imagen_portada" => $rutaImagenPortada,
                "archivador_imagenes" => $rutaArchivadorImagenes,
                "tipologia" => $tipologia,
                "modus_operandi" => $modusOperandi,
                "zona" => $zona,
                "id_usuario" => $idUsuario
            ]);
            $titulo = "CRIMEN AGREGADO EXITOSAMENTE";
            $resultado = "El crimen con titulo '$tituloCrimen' ha sido creado correctamente en la base de datos. ";
            $codigo = 0;
        }
        //Obtenemos el nºidentificativo para la gestión de ficheros (cada carpeta será un nº autoincremental como el id_crimen)
        $idCrimen = DB::table("crimenes")->max("id");

        //Comprobamos si se subió una imagen, de ser así la almacenamos en su respectiva carpeta imagenesPortada/id_crimen
        if ($request->hasFile('imagenPortada')) {

            $rutaImagenPortada = $request->file('imagenPortada')->store('imagenesPortada/' . $idCrimen, 'public');
            DB::update(
                'update crimenes
                  set imagen_portada = :imagen_portada where id = :id '
                ,
                ["imagen_portada" => $rutaImagenPortada, "id" => $idCrimen]
            );
        }

        //Comprobamos si se subieron imágenes al drag&drop, de ser así la almacenamos en su respectiva carpeta archivadorImagenes/id_crimen
        if ($request->hasFile('archivadorImagenes')) {

            $imagenes = $request->file('archivadorImagenes');
            $rutaArchivadorImagenes = "archivadorImagenes/" . Storage::disk('public')->makeDirectory('archivadorImagenes/' . $idCrimen);
            DB::update(
                'update crimenes
                  set archivador_imagenes = :archivador_imagenes where id = :id '
                ,
                ["archivador_imagenes" => $rutaArchivadorImagenes, "id" => $idCrimen]
            );

            foreach ($imagenes as $imagen) {
                $imagen->store('archivadorImagenes/' . $idCrimen, 'public');
            }
        }

        //Devolvemos la respuesta recibida al usuario en JSON para que JS trate los datos.
        return response()->json([
            'redirect_url' => route("crear-crimen", [
                'winterhorn' => session("token"),
                "resultado" => $resultado,
                "codigo" => $codigo,
                "titulo" => $titulo
            ])
        ]);
    }
```

Método editar()

```php
//Método para editar un crimen con los datos recibidos por editar crimen seguido
public function editar(Request $request)
{
    //Se obtienen y filtran los datos del formulario y de sesión
    $id = trim($request->input("id_crimen"));
    $tituloCrimen = trim($request->input("tituloCrimen"));
    $descripcion = trim($request->input("descripcion"));
    $fechaDenuncia = $request->input("fechaDenuncia");
    $tipologia = trim($request->input("tipologia"));
    $modusOperandi = trim($request->input("modusOperandi"));
    $zona = trim($request->input("zona"));
    $idUsuario = session("usuario");

    //Se comprueba si todos los id_crimen existen actualmente en la BD
    $rutaImagenPortada = DB::table("crimenes")->where('id', $id)->value('imagen_portada');
    $crimenExiste = DB::table("crimenes")->where("id", $id)->exists();
    $crimenExistente = DB::table("crimenes")->where("titulo", $tituloCrimen)->first();
    $resultado = "";
    $type = DB::select(DB::raw("SHOW COLUMNS FROM crimenes WHERE Field = 'tipologia'"))->type;
    preg_match("/^enum\('(.*)'\)$/", $type, $matches);
    $tipologiasPermitidas = explode("','", $matches[1]);

    //Validamos los datos y almacenamos el error/exito de la validación
    if (!$crimenExiste) {
        $titulo = "ERROR: EL CRIMEN NO EXISTE";
        $resultado = "Por favor, introduzca un crimen que exista en la base de datos. ";
        $codigo = 1;
    } else if (strlen($tituloCrimen) <= 0 || strlen($tituloCrimen) > 100) {
        $titulo = "ERROR: TITULO DE CRIMEN NO VALIDO";
        $resultado = "Por favor, introduzca un titulo de crimen cuyos caracteres estén en el rango indicado: [1 - 100]. ";
        $codigo = 2;
    } else if (strlen($descripcion) <= 0 || strlen($descripcion) > 4000) {
        $titulo = "ERROR: DESCRIPCION NO VALIDA";
        $resultado = "Por favor, introduzca una descripción del crimen cuyos caracteres estén en el rango indicado: [1 - 4000]. ";
        $codigo = 3;
    } else if (($crimenExistente && $crimenExistente->id != $id)) {
        $titulo = "ERROR: TITULO DE CRIMEN NO VALIDO";
        $resultado = "Por favor, introduzca un titulo de crimen que no exista";
        $codigo = 3;
    } else if (!$fechaDenuncia) {
        $titulo = "ERROR: FECHA DENUNCIA NO VALIDA";
        $resultado = "Por favor, introduzca una fecha del crimen válida con el formato indicado [DD-MM-AAAA]. ";
        $codigo = 4;
    } else if (!in_array($tipologia, $tipologiasPermitidas)) {
        $titulo = "ERROR: TIPOLOGIA DE CRIMEN NO VALIDA";
        $resultado = "Por favor, introduzca una tipología del crimen válida. ";
        $codigo = 7;
    } else if (strlen($modusOperandi) <= 0 || strlen($modusOperandi) > 4000) {
        $titulo = "ERROR: TIPOLOGIA DE CRIMEN NO VALIDA";
        $resultado = "Por favor, introduzca un modus operandis cuyos caracteres estén en el rango indicado: [1 - 100]. ";
        $codigo = 5;
    } else if (strlen($zona) <= 0 || strlen($zona) > 1000) {
        $titulo = "ERROR: ZONA DE CRIMEN NO VALIDA";
        $resultado = "Por favor, introduzca una zona de crimen válida.";
        $codigo = 6;
    } else {
        //Comprobamos si se quieren eliminar imágenes o eliminar, si es así se eliminan de la carpeta
        if ($request->has('imagenesEliminadas')) {
            $imagenesEliminadas = $request->input('imagenesEliminadas');

            foreach ($imagenesEliminadas as $nombreImagen) {
                $filePath = 'archivadorImagenes/' . $id . '/' . $nombreImagen;
                if (Storage::disk('public')->exists($filePath)) {
                    Storage::disk('public')->delete($filePath);
                }
            }
        }

        if (count(Storage::disk('public')->files('archivadorImagenes/' . $id)) < 1) {
            Storage::disk('public')->deleteDirectory('archivadorImagenes/' . $id);
            Storage::disk('public')->makeDirectory('archivadorImagenes/' . $id);

        }

        //Comprobamos si se quiere una imagen, de ser así eliminamos la anterior y almacenamos la nueva
        if ($request->hasFile('imagenPortada')) {
            $oldPortada = DB::table("crimenes")->where('id', $id)->value('imagen_portada');
            if ($oldPortada) {
                Storage::disk('public')->delete($oldPortada);
            }

            $rutaImagenPortada = $request->file('imagenPortada')->store('imagenesPortada/' . $id, 'public');
        }

        //Comprobamos si se quieren imagenes nuevas al archivador, si es así se almacenan en su carpeta y se pasan técnicos no deseados
        if ($request->hasFile('archivadorImagenes')) {

            $imagenes = $request->file('archivadorImagenes');

            foreach ($imagenes as $imagen) {
                if ($imagen->isValid() && str_starts($imagen->getMimeType(), 'image/') === 0) {

                    $extension = $imagen->getClientOriginalExtension();

                    if (in_array($extension, ['jpeg', 'png', 'jpg', 'gif'])) {
                        try {
                            $imagen->store('archivadorImagenes/' . $id, 'public');
                        } catch (\Exception $e) {
                            \Log::error('Error al almacenar la imagen: ' . $e->getMessage());
                        }

                    }
                }
            }

            $archivos = Storage::disk('public')->files('archivadorImagenes/' . $id);
            foreach ($archivos as $archivo) {
                if (pathinfo($archivo, PATHINFO_EXTENSION) === 'txt') {
                    Storage::disk('public')->delete($archivo);
                }
            }

            if (count(Storage::disk('public')->files('archivadorImagenes/' . $id)) < 1) {
                Storage::disk('public')->deleteDirectory('archivadorImagenes/' . $id);
            }

        }

        //Si no pasa errores, insertamos los datos en la BD
        $resultado = DB::update(
            'update crimenes
             set titulo = :titulo, descripcion = :descripcion, fecha_denuncia = :fecha_denuncia, imagen_portada = :imagen_portada, tipologia = :tipologia, modus_operandi = :modus_operandi, zona = :zona where id = :id  ',
            [
                'titulo' => $tituloCrimen,
                'descripcion' => $descripcion,
                'fecha_denuncia' => $fechaDenuncia,
                'imagen_portada' => $rutaImagenPortada,
                'tipologia' => $tipologia,
                'modus_operandi' => $modusOperandi,
                'zona' => $zona,
                'id' => $id
            ]
        );

        $titulo = "EXITO";
        $resultado = "Crimen editado exitosamente.";
        $codigo = 0;
    }
    //Recuperamos los datos de nuevo y los mandamos en JSON para que la vista JS
    $datosEditar = DB::table("crimenes")->select("*")->where("id", $id)->first();

    return response()->json([
        'redirect_url' => route("editar-crimen", [
            'winterhorn' => session("token"),
            'id_crimen' => $datosEditar->id,
            'tituloCrimen' => $datosEditar->titulo,
            'descripcion' => $datosEditar->descripcion,
            'fecha_denuncia' => $datosEditar->fecha_denuncia,
            'imagen_portada' => $datosEditar->imagen_portada,
            'archivador_imagenes' => $datosEditar->archivador_imagenes,
            'tipologia' => $datosEditar->tipologia,
            'tipologias' => $tipologiasPermitidas,
            'modus_operandi' => $datosEditar->modus_operandi,
            'zona' => $datosEditar->zona,
            'redactor' => $datosEditar->id_usuario,
            "codigo" => $codigo,
            "titulo" => $titulo,
            "resultado" => $resultado,
        ])
    ]);
}
```

Métodos de obtención de datos: get_crimen(), get_tipologias(), datos_editar(), datos_leer()

```php

    //Obtiene los datos del crimen para darselos a /editar-crimen-cargadb y que blade los muestre
    public function datos_editar(Request $request)
    {
        $datos = DB::table("crimenes")->select("*")->where("id", $request->get("id_crimen"))->first();
        $type = DB::selectOne("SHOW COLUMNS FROM crimenes WHERE Field = 'tipologia' ")->Type;
        preg_match("/^enum\('(.*)'\)$/", $type, $matches);
        $tipologiasPermitidas = explode("','", $matches[1]);

        return redirect()->route("editar-crimen", [
            'winterhorn' => session("token"),
            'id_crimen' => $datos->id,
            'tituloCrimen' => $datos->titulo,
            'descripcion' => $datos->descripcion,
            'fecha_denuncia' => $datos->fecha_denuncia,
            'imagen_portada' => $datos->imagen_portada,
            'archivador_imagenes' => $datos->archivador_imagenes,
            'tipologia' => $datos->tipologia,
            'tipologias' => $tipologiasPermitidas,
            'modus_operandi' => $datos->modus_operandi,
            'zona' => $datos->zona,
            'redactor' => $datos->id_usuario,
        ]);
    }

    //Obtiene los datos del crimen para darselos a /editar-crimen-cargadb y que blade los muestre
    public function datos_leer(Request $request)
    {
        $datos = DB::table("crimenes")->select("*")->where("id", $request->get("id_crimen"))->first();
        return redirect()->route("leer-crimen", [
            'winterhorn' => session("token"),
            'id_crimen' => $datos->id,
            'titulo' => $datos->titulo,
            'descripcion' => $datos->descripcion,
            'fecha_denuncia' => $datos->fecha_denuncia,
            'imagen_portada' => $datos->imagen_portada,
            'archivador_imagenes' => $datos->archivador_imagenes,
            'tipologia' => $datos->tipologia,
            'modus_operandi' => $datos->modus_operandi,
            'zona' => $datos->zona,
            'redactor' => $datos->id_usuario,
        ]);
    }

    //Obtiene los datos del crimenes para mostrarlos en /get-crimen
    public function get_crimen(Request $request)
    {

        $pagina = $request->input('pagina', 1);
        $registrosPorPagina = 3; //Cambiar a 15 en PRODUCCION

        $salto = ($pagina - 1) * $registrosPorPagina;
        $crimenes = DB::table("crimenes")->skip($salto)->take($registrosPorPagina)->get();
        $totalCrimenes = DB::table("crimenes")->count();
        $totalPaginas = ceil($totalCrimenes / $registrosPorPagina);

        return view("listado-crimen", [
            'winterhorn' => session("token"),
            "crimenes" => $crimenes,
            "pagina" => $pagina,
            "registrosPorPagina" => $registrosPorPagina,
            "totalPaginas" => $totalPaginas,
        ]);
    }

    //Obtiene las tipologías de los crímenes para el select en /crear-crimen
    public function get_tipologias()
    {
        $type = DB::selectOne("SHOW COLUMNS FROM crimenes WHERE Field = 'tipologia' ")->Type;
        preg_match("/^enum\('(.*)'\)$/", $type, $matches);
        $tipologias = explode("','", $matches[1]);

        return view('crear-crimen', [
            'winterhorn' => session("token"),
            'tipologias' => $tipologias
        ]);
    }

codesnap.dev
```

Método borrar()

```php
//Borra el crimen y las carpetas y ficheros asociados al mismo viniendo de /borrar-crimen
public function borrar(Request $request)
{
    //Obtenemos el id del crimen a borrar y comprobamos si existe en la DB
    $resultado = "";
    $id_crimen = $request->input("id_crimen");
    $crimenExiste = DB::table("crimenes")->where("id", $id_crimen)->exists();

    //Si no existe, mandamos de vuelta el mensaje de error
    if (!$crimenExiste) {
        $titulo = "ERROR: EL CRIMEN NO EXISTE";
        $resultado = "El crimen no existe en la base de datos o ya ha sido borrado.";
        $codigo = 1;
    } else { //Si existe, borramos todo sobre el crimen (carpetas de imágenes, su registro en la DB y la incidencia asociada al mismo)
        $titulo = "ÉXITO";
        Storage::deleteDirectory('public/archivadorImagenes/' . $id_crimen);
        Storage::deleteDirectory('public/imagenesPortada/' . $id_crimen);
        DB::table("incidencias")->where("id_crimen", $id_crimen)->delete();
        $resultado = DB::table("crimenes")->where("id", $id_crimen)->delete();
        $codigo = 0;
        if ($resultado) {
            $resultado = "Crimen borrado existosamente.";
        }
    }

    //Retornamos el resultado a /get-crimen
    return redirect()->route("get-crimen", [
        'winterhorn' => session("token"),
        "resultado" => $resultado,
        "codigo" => $codigo,
        "titulo" => $titulo,
        "ventana" => true
    ]);
}
```

codesnap.dev

Fichero UsuarioController.php

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Routing\Controller;
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
use Illuminate\Foundation\Validation\ValidatesRequests;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Str;
use Carbon\Carbon;
use \DateTimeZone;
/**
 *Controlador para gestionar los usuarios
 */
class UsuarioController extends Controller
{
    use AuthorizesRequests, ValidatesRequests;

```

codesnap.dev

Método entrar()

```php
//Método para iniciar sesión en la aplicación /login, asignar el token de sesión y/o bloquear al usuario
public function entrar(Request $request)
{
    //Se limpia y obtiene el formulario
    $usuario = trim($request->input("Usuario"));
    $hashPassword = $request->input("Contraseña");

    //Se comprueba si existe un usuario con ese nombre de usuario
    $usuarioDB = DB::table("usuarios")
        ->select("nombre", "password", "id", "permisos", "intentos", "tiempo_bloqueado")
        ->where("nombre", $usuario)
        ->first();

    if ($usuarioDB) {
        //Si existe, se comprueban sus password hasheadas
        if (Hash::check($hashPassword, $usuarioDB->password)) {
            //Se reinician los intentos y el timpo de bloqueo
            DB::table('usuarios')
                ->where('id', $usuarioDB->id)
                ->update(['intentos' => 0, 'tiempo_bloqueado' => null]);

            //Se genera el token de sesión y se asignan los permisos en función de su nº
            $token = Hash::make(Str::random(60));
            session(["token" => $token]);
            session(["usuario" => $usuarioDB->id]);
            switch ($usuarioDB->permisos) {
                case "40":
                case "50":
                    session(["ver" => "todo"]);
                    break;
                case "30":
                    session(["ver" => "moderador"]);
                    break;
                case "20":
                    session(["ver" => "editor"]);
                    break;
                case "10":
                    session(["ver" => "lector"]);
                    break;
                default:
                    session(["ver" => "nada"]);
                    break;
            }
            return redirect()->route("get-crimen", ['winterhorn' => $token]);
        } else {
            //Si el intento de sesión falla, se inicializa/aumenta el nº intentos
            $intentos = $usuarioDB->intentos ?? 0;
            $intentos++;
            $tiempo_bloqueado = $usuarioDB->tiempo_bloqueado ? Carbon::parse($usuarioDB->tiempo_bloqueado) : Carbon::now();

            //Si el usuario superó los 10 intentos, se establece el tiempo de bloqueo en 2h
            if ($intentos > 10) {

                $tiempo_transcurrido = Carbon::now()->diffInSeconds($tiempo_bloqueado);

                if ($tiempo_transcurrido < 7200) {
                    $tiempo_restante = 7200 - $tiempo_transcurrido;
                    $horas = intdiv($tiempo_restante, 3600);
                    $minutos = intdiv($tiempo_restante % 3600, 60);
                    //Cada vez que trate de iniciar sesión, si no pasó el tiempo de bloqueo, se le indica y se bloquea el acceso
                    return back()->with([
                        'error' => "Has superado el límite de intentos de inicio de sesión. ",
                        'tiempoEspera' => "$horas horas y $minutos minutos"
                    ]);
                } else {
                    //Si ya pasó el tiempo de bloqueo, se le desbloquea
                    $intentos = 0;
                    $tiempo_bloqueado = Carbon::now()->addHours(2);
                }
            }
            //Se actualiza en la DB los intentos y tiempo de bloqueo
            DB::table('usuarios')
                ->where('id', $usuarioDB->id)
                ->update([
                    'intentos' => $intentos,
                    'tiempo_bloqueado' => $tiempo_bloqueado
                ]);

            //Se indica que las credenciales fueron erróneas
            return back()->with('error', "Usuario o contraseña inválidos. Nº Intento: $intentos");
        }
    } else {
        //Si no existe el usuario, se le indica
        return back()->with("error", "No existe el usuario indicado");
    }
}
```

codesnap.dev

Método crear()

```php
    //Método para crear un crimen con los datos recibidos por /crear-crimen
    public function crear(Request $request)
    {
        //Se obtienen y filtran los datos del formulario y de sesión
        $nombre = trim($request->input("nombre"));
        $contrasena = $request->input("contrasena");
        $permisos = $request->input("permisos");
        $creado = Carbon::now(new DateTimeZone('Europe/Madrid')); //Se pone la fecha de cuando se creó el usuario
        $creado = $creado->format('Y-m-d H:i:s');
        $actualizado = Carbon::now(new DateTimeZone('Europe/Madrid')); //Siempre se actualiza el tiempo de actualización (como un touch)
        $actualizado = $actualizado->format('Y-m-d H:i:s');

        //Validamos los datos y generamos el texto de éxito/error
        $resultado = "";
        $usuarioExiste = DB::table("usuarios")->where("nombre", $nombre)->exists();
        if ($usuarioExiste) {
            $titulo = "ERROR: EL USUARIO YA EXISTE";
            $resultado = "Por favor, introduzca un usuario que no exista en la base de datos. ";
            $codigo = 1;
        } else if (strlen($nombre) <= 0 || strlen($nombre) > 45) {
            $titulo = "ERROR: NOMBRE DE USUARIO NO VALIDO";
            $resultado = "Por favor, introduzca un nombre de usuario cuyos caracteres estén en el rango indicado: [1 - 45]. ";
            $codigo = 2;
        }  else if ($permisos != 10 && $permisos != 20 && $permisos != 30 && $permisos != 40 && $permisos != 50) {
            $titulo = "ERROR: PERMISOS INCORRECTOS";
            $resultado = "Por favor, introduzca los permisos adecuados. ";
            $codigo = 4;
        } else {
            //Si todo fue bien, se crea el usuario y se le indica al usuario
            $resultado = DB::table("usuarios")->insert([
                "nombre" => $nombre,
                "password" => Hash::make($contrasena),
                "permisos" => $permisos,
                "creado" => $creado,
                "actualizado" => $actualizado,
            ]);
            $titulo = "USUARIO AGREGADO EXITOSAMENTE";
            $resultado = "El usuario '$nombre' con permisos '$permisos' ha sido creado correctamente en la base de datos. ";
            $codigo = 0;
        }
        //Se reenvia al usuario el mensaje de error/exito
        return redirect()->route("crear-usuario", [
            'winterhorn' => session("token"),
            "resultado" => $resultado,
            "codigo" => $codigo,
            "titulo" => $titulo
        ]);
    }
```

codesnap.dev

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Método editar()

```php
//Método para editar un usuario con los datos recibidos por editar-usuario-cargadb
public function editar(Request $request)
{
    //Se obtienen y filtran los datos del formulario y de sesión
    $id = $request->input("id_usuario");
    $nombre = trim($request->input("nombre"));
    $contrasena = null;
    if (strlen($request->input("password")) > 30) {
        //Se comprueba la contraseña, si es correcta se genera el hash
        $contrasena = $request->input("password");
    } else {
        $contrasena = Hash::make($request->input("password"));
    }
    $permisos = $request->input("permisos");
    $creado = $request->input("creado");
    $actualizado = Carbon::now(new DateTimeZone('Europe/Madrid')); //Siempre se actualiza el tiempo de actualizado (como un touch)
    $actualizado = $actualizado->format('Y-m-d H:i:s');
    $usuarioExiste = DB::table("usuarios")->where("id", $id)->exists();
    $resultado = "";

    //Validamos los datos y almacenamos el error/éxito de la validación
    if (!$usuarioExiste) {
        $titulo = "ERROR: EL USUARIO NO EXISTE";
        $resultado = "Por favor, introduzca un usuario que exista en la base de datos. ";
        $codigo = 1;
    } else if (strlen($nombre) <= 0 || strlen($nombre) > 45) {
        $titulo = "ERROR: NOMBRE DE USUARIO NO VALIDO";
        $resultado = "Por favor, introduzca un nombre de usuario cuyos caracteres estén en el rango indicado: [1 - 45]. ";
        $codigo = 2;
    } else if ($permisos != 10 && $permisos != 20 && $permisos != 30 && $permisos != 40 && $permisos != 50) {
        $titulo = "ERROR: PERMISOS INCORRECTOS";
        $resultado = "Por favor, introduzca los permisos adecuados. ";
        $codigo = 4;
    } else {
        //Si todo va bien, actualizamos el usuario
        $resultado = DB::update(
            'update usuarios
              set nombre = :nombre, password = :password, permisos = :permisos, actualizado = :actualizado
              where id = :id',
            ['nombre' => $nombre, 'password' => $contrasena, 'permisos' => $permisos, 'actualizado' => $actualizado, 'id' => $id]
        );

        $titulo = "ÉXITO";
        $resultado = "Usuario editado exitosamente.";
        $codigo = 0;
    }
    $datoseditar = DB::table("usuarios")->select("*")->where("id", $id)->first();

    //Devolvemos al usuario si se editó o no correctamente el usuario
    return redirect()->route("editar-usuario", [
        'winterhorn' => session("token"),
        'id_usuario' => $datoseditar->id,
        'nombre' => $datoseditar->nombre,
        'password' => $datoseditar->password,
        'permisos' => $datoseditar->permisos,
        'creado' => $datoseditar->creado,
        'actualizado' => $datoseditar->actualizado,
        "codigo" => $codigo,
        "titulo" => $titulo,
        "resultado" => $resultado,
    ]);
}
```

codesnap.dev

y Christian Miguel Giménez Puerma

Métodos de obtención de datos: get_usuario() y datos_leer()

```php
//Obtiene los datos del usuario para darselos a /editar-usuario-cargadb y que blade los muestre
public function datos_editar(Request $request)
{
    $datos = DB::table("usuarios")->select("*")->where("id", $request->get("id_usuario"))->first();

    return redirect()->route("editar-usuario", [
        'winterhorn' => session("token"),
        'id_usuario' => $datos->id,
        'nombre' => $datos->nombre,
        'password' => $datos->password,
        'permisos' => $datos->permisos,
        'creado' => $datos->creado,
        'actualizado' => $datos->actualizado,
    ]);
}

//Obtiene los datos del usuario para mostrarlos en /get-usuario
public function get_usuario(Request $request)
{
    //Sistema de paginación y mostrar la ordenación
    $pagina = $request->input('pagina', 1);
    $ordenarPor = $request->input('ordenarPor', 'nombre');
    $direccion = $request->input('direccion', 'asc');
    $registrosPorPagina = 15; //Cambiar a 15 en PRODUCCION

    $salto = ($pagina - 1) * $registrosPorPagina;
    $usuarios = DB::table("usuarios")->orderBy($ordenarPor, $direccion)->skip($salto)->take($registrosPorPagina)->get();

    if (request()->has("buscar")) {
        $buscar = request()->input("buscar");

        $usuarios = DB::table("usuarios")->orderBy($ordenarPor, $direccion)->skip($salto)->take($registrosPorPagina)
            ->where("nombre", "like", "%" . $buscar . "%")
            ->orWhere("permisos", "like", "%" . $buscar . "%")
            ->get();
    }



    $totalUsuarios = DB::table("usuarios")->count();
    $totalPaginas = ceil($totalUsuarios / $registrosPorPagina);

    //Mandamos al usuario los datos, orden, paginas totales y pagina actual para pintarlos con blade
    return view("listado-usuario", [
        'winterhorn' => session("token"),
        "usuarios" => $usuarios,
        "pagina" => $pagina,
        "registrosPorPagina" => $registrosPorPagina,
        "totalPaginas" => $totalPaginas,
        "ordenarPor" => $ordenarPor,
        "direccion" => $direccion
    ]);
}
```

codesnap.dev

y Christian Miguel Giménez Puerma

Método borrar()

```php
    //Borra el usuario
    public function borrar(Request $request)
    {
        //Obtenemos el id del usuario a borrar y comprobamos si existe en la DB
        $resultado = "";
        $idUsuario = $request->input("id_usuario");
        $usuarioExiste = DB::table("usuarios")->where("id", $idUsuario)->exists();

        //Si no existe, mandamos el error
        if (!$usuarioExiste) {
            $titulo = "ERROR: EL USUARIO NO EXISTE";
            $resultado = "El usuario no existe en la base de datos o ya ha sido borrado. ";
            $codigo = 1;
        } else {
            //Si existe, lo borramos y mandamos el éxito
            $titulo = "ÉXITO";
            $resultado = DB::table("usuarios")->where("id", $idUsuario)->delete();
            $codigo = 0;
            if ($resultado) {
                $resultado = "Usuario borrado existosamente.";
            }
        }

        return redirect()->route("get-usuario", [
            'winterhorn' => session("token"),
            "resultado" => $resultado,
            "codigo" => $codigo,
            "titulo" => $titulo,
            "borrado" => true
        ]);

    }
```

codesnap.dev

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Fichero IncidenciaController.php

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Routing\Controller;
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
use Illuminate\Foundation\Validation\ValidatesRequests;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Str;
use Carbon\Carbon;
use \DateTimeZone;

/**
 *Controlador para gestionar las incidencias
 */
class IncidenciaController extends Controller
{
    use AuthorizesRequests, ValidatesRequests;

    //Método para crear una incidencia asociada a un usuario y un crimen
    public function crear(Request $request)
    {
        //Se obtienen y filtran los datos del formulario y de sesión
        $id_redactor = $request->input('redactor');
        $id_crimen = $request->input('id_crimen');
        $resultado = "";
        $incidenciaExiste = DB::table("incidencias")->where("id_usuario", $id_redactor)->where("id_crimen", $id_crimen)->exists();

        //Si la incidencia no existía, se crea, si no, se manda error
        if (!$incidenciaExiste) {
            DB::table("incidencias")->insert([
                "id_usuario" => $id_redactor,
                "id_crimen" => $id_crimen,
            ]);
            $titulo = "INCIDENCIA CREADA EXITOSAMENTE";
            $resultado = "INCIDENCIA CREADA EXITOSAMENTE";
            $codigo = 0;
        } else {
            $titulo = "ERROR";
            $resultado = "ERROR: INCIDENCIA YA EXISTENTE";
            $codigo = 1;
        }

        //Se reenvia al usuario el mensaje de error/exito
        return redirect()->route("get-crimen", [
            'winterhorn' => session("token"),
            "resultado" => $resultado,
            "codigo" => $codigo,
            "titulo" => $titulo,
            'ventana' => true
        ]);
    }

    //Obtiene los datos de la incidencia para mostrarla en /get-incidencia
    public function get_incidencia(Request $request)
    {
        //Sistema de paginacion y mostrar la ordenacion
        $pagina = $request->input('pagina', 1);
        $ordenarPor = $request->input('ordenarPor', 'id');
        $direccion = $request->input('direccion', 'asc');
        $registrosPorPagina = 15; //Cambiar a 15 en PRODUCCION

        $salto = ($pagina - 1) * $registrosPorPagina;
        $incidencias = DB::table("incidencias")->orderBy($ordenarPor, $direccion)->skip($salto)->take($registrosPorPagina)->get();


        if (request()->has("buscar")) {
            $buscar = request()->input("buscar");
            $incidencias = DB::table("incidencias")->orderBy($ordenarPor, $direccion)->skip($salto)->take($registrosPorPagina)
            ->where("id", "like", "%" . $buscar . "%")
            ->orWhere("id_crimen", "like", "%" . $buscar . "%")
            ->orWhere("id_usuario", "like", "%" . $buscar . "%")
            ->get();
        }

        $totalIncidencias = DB::table("incidencias")->count();
        $totalPaginas = ceil($totalIncidencias / $registrosPorPagina);

        //Mandamos al usuario los datos, orden, paginas totales y pagina actual para pintarlos con blade
        return view("listado-incidencia", [
            'winterhorn' => session("token"),
            "incidencias" => $incidencias,
            "pagina" => $pagina,
            "registrosPorPagina" => $registrosPorPagina,
            "totalPaginas" => $totalPaginas,
            "ordenarPor" => $ordenarPor,
            "direccion" => $direccion
        ]);
    }



    //Borra la incidencia
    public function borrar(Request $request)
    {
        //Obtenemos el id de la incidencia a borrar y comprobamos si existe en la DB
        $resultado = "";
        $idIncidencia = $request->input("id_incidencia");
        $incidenciaExiste = DB::table("incidencias")->where("id", $idIncidencia)->exists();

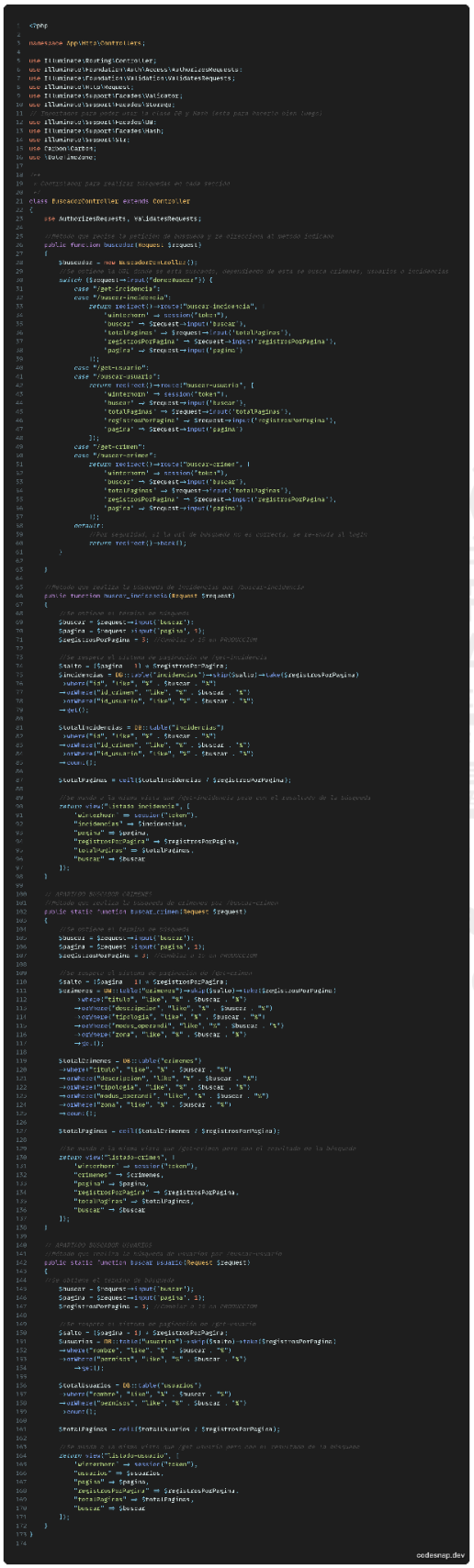
        //Si no existe, mandamos el error
        if (!$incidenciaExiste) {
            $titulo = "ERROR: LA INCIDENCIA NO EXISTE";
            $resultado = "La incidencia no existe en la base de datos o ya ha sido borrado. ";
            $codigo = 1;
        } else {
            //Si existe, la borramos y mandamos el éxito
            $titulo = "EXITO";
            $resultado = DB::table("incidencias")->where("id", $idIncidencia)->delete();
            $codigo = 0;
            if ($resultado) {
                $resultado = "Incidencia borrada existosamente. ";
            }
        }

        return redirect()->route("get-incidencia", [
            'winterhorn' => session("token"),
            "resultado" => $resultado,
            "codigo" => $codigo,
            "titulo" => $titulo,
            "borrado" => true
        ]);
    }
}

```

codesnap.dev

Fichero BuscadorController.php



Código módulos de JavaScript

Fichero crimenes.js

```js
import { crearApendarNodos } from "./app.js";

//Pone los eventos a las cards para redireccionar al enlace pertinente
document.querySelectorAll(".card-crimen").forEach(card => {
    card.style.cursor = "pointer";
    let enlace = card.querySelector(".txt-crimen > a").getAttribute("href");
    card.addEventListener("click", function(event) {
        event.stopPropagation();
        location.href = enlace;
    })
});
```

codesnap.dev

Fichero usuarios.js

```js
import { crearApendarNodos } from "./app.js";

//Recogemos las variables HTTP para realizar la peticion en JS
const urlParams = new URLSearchParams(window.location.search);
const token = urlParams.get('winterhorn');
const pagina = urlParams.get('pagina') || 1;
const buscar = urlParams.get('buscar');
const registrosPorPagina = urlParams.get('registrosPorPagina');
const totalPaginas = urlParams.get('totalPaginas');

//Creamos la tabla de usuarios
usuarios.forEach(function(usuario) {
    const tr = crearApendarNodos(document.querySelector('table tbody'), 'tr');

    crearApendarNodos(tr, 'td', {}, usuario.nombre);
    crearApendarNodos(tr, 'td', {}, usuario.permisos);
    crearApendarNodos(tr, 'td', {}, usuario.creado);
    crearApendarNodos(tr, 'td', {}, usuario.actualizado);

    const td = crearApendarNodos(tr, 'td');

    //Añadimos el enlace para llegar al controlador que edita el usuario
    const enlace1 = crearApendarNodos(td, 'a', {
        href: '/editar-usuario-cargado?winterhorn=' + token + '&id_usuario=' + usuario.id
    });
    crearApendarNodos(enlace1, 'img', {
        src: '../imgs/lapiz.png',
        alt: 'Editar usuario'
    });

    //Añadimos el enlace para llegar al controlador que borra el usuario
    const enlace2 = crearApendarNodos(td, 'a', {
        href: '/borrar-usuario?winterhorn=' + token + '&id_usuario=' + usuario.id
    });
    crearApendarNodos(enlace2, 'img', {
        src: '../imgs/basura.png',
        alt: 'Borrar usuario'
    });
});

//Guardamos el estado de las columnas para saber por cual quiere el usuario ordenar
const columnas = {
    'USUARIO': 'nombre',
    'PERMISOS': 'permisos',
    'FECHA CREACIÓN': 'creado',
    'FECHA ACTUALIZACIÓN': 'actualizado'
};

//Recuperamos el orden actual por las variables HTTP
let ordenActual = {
    columna: urlParams.get('ordenarPor') || 'nombre',
    direccion: urlParams.get('direccion') || 'asc'
};

//Añadimos los eventos para cambiar de orden las columnas
document.querySelectorAll('th').forEach((th, i, arr) => {
    th.dataset.orden = columnas[th.textContent];
    if (i < arr.length - 1) {
        th.addEventListener('click', () => {
            //Por defecto el orden es
            if (ordenActual.columna === th.dataset.orden) {
                ordenActual.direccion = ordenActual.direccion === 'asc' ? 'desc' : 'asc';
            } else {
                ordenActual.direccion = 'asc';
            }

            var nuevaUrl = '';
            //En función de la columna clickeada y su estado (asc o desc) se genera una petición para reordenar los usuarios por el término indicado
            if (buscar) {
                nuevaUrl = `/get-usuario?winterhorn=${token}&pagina=${pagina}&buscar=${buscar}&registrosPorPagina=${registrosPorPagina}&totalPaginas=${totalPaginas}&ordenarPor=${th.dataset.orden}&direccion=${ordenActual.direccion}`;
            }else{
                nuevaUrl = `/get-usuario?winterhorn=${token}&pagina=${pagina}&ordenarPor=${th.dataset.orden}&direccion=${ordenActual.direccion}`;
            }

            window.location.href = nuevaUrl;
        });
    }
});
```

codesnap.dev

Fichero incidencias.js



```
import { crearApendarNodos } from "./app.js";

//Recogemos las variables HTTP para realizar la navegación en JS
const urlParams = new URLSearchParams(window.location.search);
const token = urlParams.get('winterhorn');
const pagina = urlParams.get('pagina') || 1;
const buscar = urlParams.get('buscar');
const registrosPorPagina = urlParams.get('registrosPorPagina');
const totalPaginas = urlParams.get('totalPaginas');

//Creamos la tabla de incidencias
incidencias.forEach(function (incidencia) {
    const tr = crearApendarNodos(document.querySelector('table tbody'), 'tr');

    crearApendarNodos(tr, 'td', {}, incidencia.id);
    crearApendarNodos(tr, 'td', {}, incidencia.id_usuario);
    crearApendarNodos(tr, 'td', {}, incidencia.id_crimen);

    const a = crearApendarNodos(tr, 'td');

    //Añadimos el enlace para llegar al controlador que muestra la lectura del crimen asociado a esa incidencia
    const enlace1 = crearApendarNodos(a, 'a', {
        href: '/leer-crimen-db?winterhorn=' + token + '&id_crimen=' + incidencia.id_crimen
    });
    crearApendarNodos(enlace1, 'img', {
        src: '../imgs/compartir-enlace.png',
        alt: 'IR AL CRIMEN'
    });

    //Añadimos el enlace para llegar al controlador que borra la incidencia
    const enlace2 = crearApendarNodos(a, 'a', {
        href: '/borrar-incidencia?winterhorn=' + token + '&id_incidencia=' + incidencia.id
    });
    crearApendarNodos(enlace2, 'img', {
        src: '../imgs/basura.png',
        alt: 'Borrar incidencia'
    });
});

//Guardamos el estado de las columnas para saber por cual quiere el usuario ordenar
const columnas = {
    'ID': 'id',
    'REDACTOR CRIMEN': 'id_usuario',
    'NOMBRE CRIMEN': 'id_crimen'
};

//Recuperamos el orden actual por las variables HTTP
let ordenActual = {
    columna: urlParams.get('ordenarPor') || 'id',
    direccion: urlParams.get('direccion') || 'asc'
};

//Añadimos los selectores para cambiar de orden las columnas
document.querySelectorAll('th').forEach((th, i, arr) => {
    th.dataset.orden = columnas[th.textContent];
    if (i < arr.length - 1) {
        th.addEventListener('click', () => {
            //Si pinchamos en la misma columna
            if (ordenActual.columna == th.dataset.orden) {
                ordenActual.direccion = ordenActual.direccion == 'asc' ? 'desc' : 'asc';
            } else {
                ordenActual.direccion = 'asc';
            }

            var nuevaUrl = '';
            //En función de la columna clickeada y su estado (asc o desc) se genera una petición para reordenar las incidencias por el término indicado
            if (buscar) {
                nuevaUrl = `/get-incidencia?winterhorn=${token}&pagina=${pagina}&buscar=${buscar}&registrosPorPagina=${registrosPorPagina}&totalPaginas=${totalPaginas}&ordenarPor=${th.dataset.orden}&direccion=${ordenActual.direccion}`;

            } else {
                nuevaUrl = `/get-incidencia?winterhorn=${token}&pagina=${pagina}&ordenarPor=${th.dataset.orden}&direccion=${ordenActual.direccion}`;
            }
            window.location.href = nuevaUrl;
        });
    }
});
```

codesnap.dev

Fichero menu.js

```
//Necesario para re-organizar el menú navbar al redimensionar a versión móvil
let boton = document.querySelector(".boton_menu_movil");

//Añadimos el evento para hacer toggle e ir ocultando/mostrando los elementos al redimensionar
boton.addEventListener("click",function() {
    let menu = document.getElementById("menu");
    menu.classList.toggle("menu_slide_oculto");
    let img = document.getElementById("imagen_inicial_navbar");
    let h1 = document.getElementById("titulo_movil");
    if(h1)h1.classList.toggle("oculto");

    //Si estamos en movil, modificamos los estilos del logo
    if(img.src.includes("CCC-logo-mobile")){
        img.src = "../imgs/LOGO_DEFINITIVO.png";
        img.style.width = "55px";
        img.style.height = "55px";
    }else{
        img.src = "../imgs/CCC-logo-mobile.png";
        img.style.width = "100px";
        img.style.height = "50px";
    }
})
```

codesnap.dev

Proyecto 2ºDAW. Autores:

Fichero validarFormularios.js

```
import { crearApendarNodos } from "./app.js";

// Función para crear y mostrar la ventana de error con los datos recibidos del controlador (igual que en blade, pero para la validación en el front)
function mostrarVentanaError(mensaje) {
    const bloquearPantalla = crearApendarNodos(document.body, 'div', { class: 'bloquear-pantalla' });
    const ventana = crearApendarNodos(bloquearPantalla, 'div', { class: 'ventana' });

    const headerVentana = crearApendarNodos(ventana, 'header', { class: 'header-ventana' });
    crearApendarNodos(headerVentana, 'h5', {}, 'ERROR');
    const btnCerrar = crearApendarNodos(headerVentana, 'button', { class: 'btn-x' }, 'X');

    const bodyVentana = crearApendarNodos(ventana, 'div', { class: 'body-ventana' });
    crearApendarNodos(bodyVentana, 'h4', {}, 'Validación fallida');
    const figura = crearApendarNodos(bodyVentana, 'figure', { class: 'texto-ventana' });
    crearApendarNodos(figura, 'img', { class: 'img-ventana', src: './imgs/error.png' });
    crearApendarNodos(figura, 'figcaption', {}, mensaje);

    const btnOk = crearApendarNodos(bodyVentana, 'button', { class: 'btn-ok btn-login' }, 'OK');

    btnCerrar.addEventListener('click', () => bloquearPantalla.remove());
    btnOk.addEventListener('click', () => bloquearPantalla.remove());
}

//Función gestora del evento de validar formulario
function validarFormulario(event) {
    event.stopPropagation();
    const form = event.target;
    let esValido = true;
    let mensajeError = '';

    //Recorremos cada elemento del formulario para comprobar si sus datos son válidos
    Array.from(form.elements).forEach(element => {
        if (element.type === 'textarea') { //Para los textareas, validamos la longitud entre 1 y 4000
            if (element.value.length < 1 || element.value.length > 4000) {
                esValido = false;
                mensajeError = element.title || 'El texto debe contener entre 1 y 4000 caracteres.';
            } //Aquí el texto cumpliendo el pattern del html y lo testeamos, si no cumple guardamos el error según el texto del title
        } else if (element.pattern && new RegExp(element.pattern).test(element.value)) {
            esValido = false;
            mensajeError = element.title;
        }
    });

    //Si no es válido, lanzamos la ventana de error con la info del title
    if (!esValido) {
        event.preventDefault();
        mostrarVentanaError(mensajeError);
        return false;
    }
    //Si va todo bien, no mostramos la ventana, se prosigue y se manda al servidor
    return true;
}


//Guardamos en window el método validarFormulario para que sea accesible
window.validarFormulario = validarFormulario;

//Seleccionamos todos los input (salvo el buscador y el que envíe ficheros dragDrop) y ejecutamos su comprobación
document.querySelectorAll('form:not(.buscador)').forEach(form => {
    if (form.getAttribute('enctype') === 'multipart/form-data') {
        form.addEventListener('submit', (evento) => {
            evento.stopPropagation();
            const esFormularioValido = validarFormulario(evento);
            if (esFormularioValido) { //Si es válido, mandamos el formulario en un JSON al servidor
                enviarFormJSON(evento, form);
            }
        });
    }
});

//Función que permite mostrar/ocultar la password dandole al botón del ojo
function mostrarPassword(event) {
    event.preventDefault();
    //Guardamos todos los datos actuales del input
    const inputActual = document.querySelector("[data-ocultar-texto]");
    const clase = inputActual.classList.value;
    const name = inputActual.name;
    const id = inputActual.id;
    const title = inputActual.title;
    const password = inputActual.value;
    const pattern = inputActual.pattern;
    let inputNuevo;
    let ocultarTexto = inputActual.dataset.ocultarTexto;
    let type = "";

    //Si se marcó para mostrar el texto, el input pasa a ser tipo text
    if(ocultarTexto == "true") {
        type = "text";
        ocultarTexto = "false";
        event.target.style.backgroundImage = "url('../imgs/visible.png')";
    } else {//Si se marcó para ocultar el texto, el input pasa a ser tipo password
        type = "password";
        ocultarTexto = "true";
        event.target.style.backgroundImage = "url('../imgs/ojo.png')";
    }

    //Si existiría un pattern, creamos el nuevo input con pattern
    if (pattern) {
        inputNuevo = crearApendarNodos(null, "input", {
            type: type,
            "data-ocultar-texto": ocultarTexto,
            class: clase,
            name: name,
            pattern: pattern,
            id: id,
            title: title,
            value: password
        }, "");
    } else { //Si no, lo creamos sin pattern (es necesario esto, si no crearía siempre pattern vacío "" y fallaría la validación)
        inputNuevo = crearApendarNodos(null, "input", {
            type: type,
            "data-ocultar-texto": ocultarTexto,
            class: clase,
            name: name,
            id: id,
            title: title,
            value: password
        }, "");
    }

    //Apendamos el nuevo input con los datos exactos y borramos el anterior
    inputActual.parentElement.insertBefore(inputNuevo, inputActual);
    inputActual.parentElement.removeChild(inputActual);
}

//Añadimos el evento de mostrarPassword al ojo
const campoPassword = document.querySelector('.mostrarPassword');
if(campoPassword) {
    campoPassword.addEventListener("click", mostrarPassword);
}
```

codesnap.dev

Fichero dragNdrop.js

```
import { crearApendarNodos } from "./app.js";

// Función para crear y mostrar la ventana de error con los datos recibidos del controlador (igual que en blade, pero para la validación en el front)
function mostrarVentanaError(mensaje) {
    const bloquearPantalla = crearApendarNodos(document.body, 'div', { class: 'bloquear-pantalla' });
    const ventana = crearApendarNodos(bloquearPantalla, 'div', { class: 'ventana' });

    const headerVentana = crearApendarNodos(ventana, 'header', { class: 'header-ventana' });
    crearApendarNodos(headerVentana, 'h5', {}, 'ERROR');
    const btnCerrar = crearApendarNodos(headerVentana, 'button', { class: 'btn-x' }, 'X');

    const bodyVentana = crearApendarNodos(ventana, 'div', { class: 'body-ventana' });
    crearApendarNodos(bodyVentana, 'h4', {}, 'Validación fallida');
    const figura = crearApendarNodos(bodyVentana, 'figure', { class: 'texto-ventana' });
    crearApendarNodos(figura, 'img', { class: 'img-ventana', src: './imgs/error.png' });
    crearApendarNodos(figura, 'figcaption', {}, mensaje);

    const btnOk = crearApendarNodos(bodyVentana, 'button', { class: 'btn-ok btn-login' }, 'OK');

    btnCerrar.addEventListener('click', () => bloquearPantalla.remove());
    btnOk.addEventListener('click', () => bloquearPantalla.remove());
}

//Función gestora del evento de validar formulario
function validarFormulario(event) {
    event.stopPropagation();
    const form = event.target;
    let esValido = true;
    let mensajeError  = '';

    //Recorremos cada elemento del formulario para comprobar si sus datos son válidos
    Array.from(form.elements).forEach(element => {
        if (element.type === 'textarea') { //Para los textarea, validamos la longitud entre 1 y 4000
            if (element.value.length < 1 || element.value.length > 4000) {
                esValido  = false;
                mensajeError = element.title || 'El texto debe contener entre 1 y 4000 caracteres.';
            } //Para el resto, comprobamos el patrón del input y lo testeamos, si no cuadra guardamos el error según el texto del title
        } else if (element.pattern && !new RegExp(element.pattern).test(element.value)) {
            esValido = false;
            mensajeError = element.title;
        }
    });

    //Si no es válido, lanzamos la ventana de error con la info del title
    if (!esValido) {
        event.preventDefault();
        mostrarVentanaError(mensajeError);
        return false;
    }
    //Si va todo bien, no mostramos la ventana, se prosigue y se manda al servidor
    return true;
}


//Guardamos en window el método validarFormulario para que sea accesible
window.validarFormulario = validarFormulario;

//Seleccionamos todos los input (salvo el buscador y el que envíe ficheros dragDrop) y ejecutamos su comprobación
document.querySelectorAll('form:not(.buscador)').forEach(form => {
    if (form.getAttribute('enctype') !== 'multipart/form-data') {
        form.addEventListener('submit', (evento) => {
            evento.stopPropagation();
            const esFormularioValido = validarFormulario(evento);
            if (esFormularioValido) { //Si es válido, mandamos el formulario en un JSON al servidor
                enviarFormJSON(evento, form);
            }
        });
    }
});

//Función que permite mostrar/ocultar la password dandole al botón del ojo
function mostrarPassword(event) {
    event.preventDefault();
    //Cogiendo todos los datos actuales del input
    const inputActual = document.querySelector("[data-ocultar-texto]");
    const clase  = inputActual.classList.value;
    const name = inputActual.name;
    const id = inputActual.id;
    const title = inputActual.title;
    const password = inputActual.value;
    const pattern = inputActual.pattern;
    let inputNuevo;
    let ocultarTexto = inputActual.dataset.ocultarTexto;
    let type = "";

    //Si se marcó para mostrar el texto, el input pasa a ser tipo text
    if(ocultarTexto == "true") {
        type = "text";
        ocultarTexto = "false";
        event.target.style.backgroundImage = "url('../imgs/visible.png')";
    } else { //Si se marcó para ocultar el texto, el input pasa a ser tipo password
        type = "password";
        ocultarTexto = "true";
        event.target.style.backgroundImage = "url('../imgs/ojo.png')";
    }

    //Si existiera un pattern, creamos el nuevo input con pattern
    if (pattern) {
        inputNuevo = crearApendarNodos(null, "input", {
            type: type,
            "data-ocultar-texto": ocultarTexto,
            class: clase,
            name: name,
            pattern: pattern,
            id: id,
            title: title,
            value: password
        }, "");
    } else { //Si no, lo creamos sin pattern (es necesario esto, si no crearía siempre pattern vacío "" y fallaría la validación)
        inputNuevo = crearApendarNodos(null, "input", {
            type: type,
            "data-ocultar-texto": ocultarTexto,
            class: clase,
            name: name,
            id: id,
            title: title,
            value: password
        }, "");
    }

    //Apendamos el nuevo input con los datos exactos y borramos el anterior
    inputActual.parentElement.insertBefore(inputNuevo, inputActual);
    inputActual.parentElement.removeChild(inputActual);
}

//Añadimos el evento de mostrarPassword al ojo
const campoPassword = document.querySelector('.mostrarPassword');
if(campoPassword) {
    campoPassword.addEventListener("click", mostrarPassword);
}

codesnap.dev
```

Nota: Para no saturar el documento actual, el resto del código relativo a las vistas Blade y al enmaquetado CSS se puede consultar en el código fuente del aplicativo.

9. Manual de configuración y funcionamiento

Se brindará un manual de despliegue que tendrá las indicaciones requeridas para el correcto funcionamiento del aplicativo en un servidor web bajo la arquitectura LAMP, el cual se realiza a través de la ejecución de un script. No obstante, se deja aquí reflejado el proceso originario de instalación y configuración del aplicativo en el servidor utilizado para la defensa del proyecto.

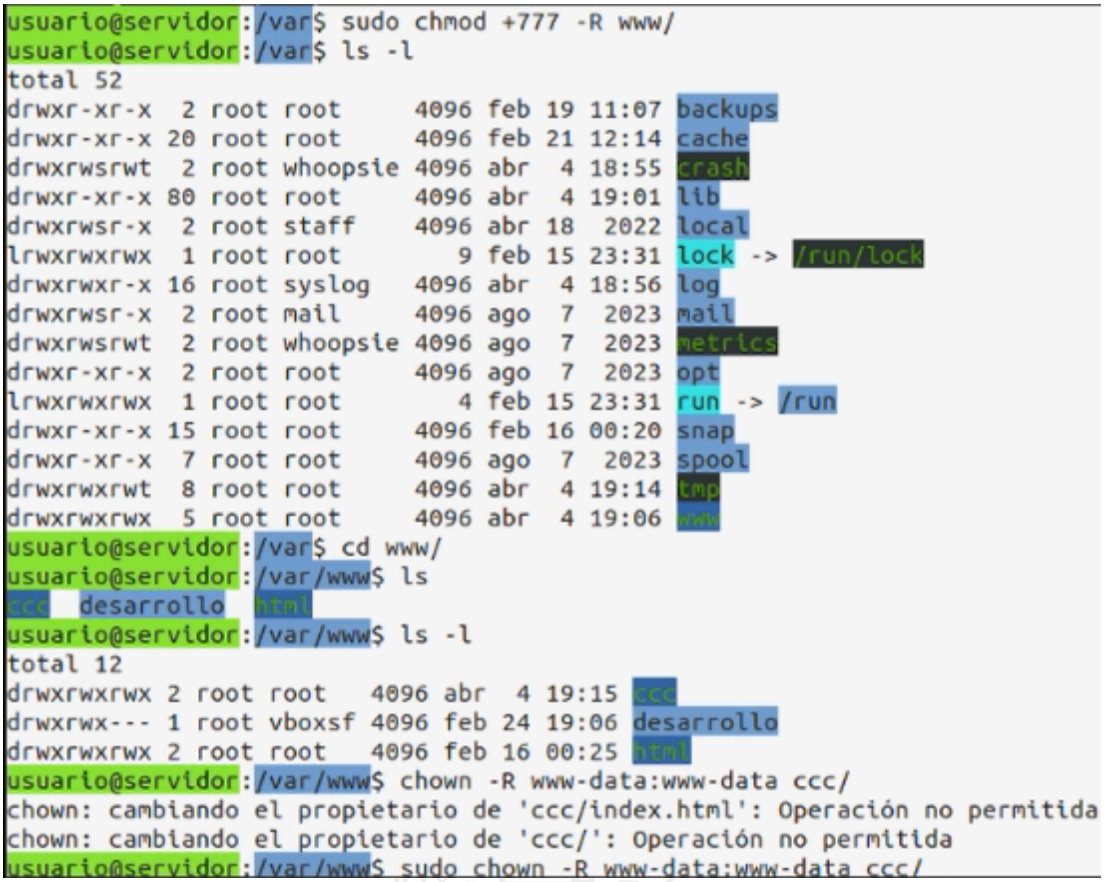
Instalación de LAMP

```
sudo wget -c https://dl.google.com/linux/direct/google-chrome-stable_current_amd64.deb
sudo apt-get update
sudo dpkg -i google-chrome-stable_current_amd64.deb
sudo snap install code --classic
sudo apt install mysql-server
sudo apt install apache2
sudo apt-get install php libapache2-mod-php php-mcrypt php-mysql
sudo apt-get install php libapache2-mod-php php-mysql
sudo apt install mysql-workbench
sudo snap install mysql-workbench-community
sudo nano /etc/apache2/mods-enabled/dir.conf
```

Configurar site y pruebas

```
nano ccc.conf
sudo nano ccc.conf
sudo a2dissite 000-default.conf
sudo a2ensite ccc.conf
clear
sudo systemctl restart apache2
echo "<h1> hello nice</h1>" > /var/www/ccc/index.html
```

Configuración de permisos



Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Instalación de Laravel

```
usuario@servidor:/var/www/ccc$ composer create-project laravel/laravel:^10.0 ccc-app
Creating a "laravel/laravel:^10.0" project at "./ccc-app"
Installing laravel/laravel (v10.3.3)
  - Installing laravel/laravel (v10.3.3): Extracting archive
Created project in /var/www/ccc/ccc-app
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies
Lock file operations: 111 installs, 0 updates, 0 removals
  - Locking brick/math (0.11.0)
  - Locking carbonphp/carbon-doctrine-types (2.1.0)
  - Locking dflydev/dot-access-data (v3.0.2)
  - Locking doctrine/inflector (2.0.10)
  - Locking doctrine/lexer (3.0.1)
  - Locking dragonmantank/cron-expression (v3.3.3)
  - Locking egulias/email-validator (4.0.2)
  - Locking fakerphp/faker (v1.23.1)
  - Locking filp/whoops (2.15.4)
  - Locking fruitcake/php-cors (v1.3.0)
  - Locking graham-campbell/result-type (v1.1.2)
  - Locking guzzlehttp/guzzle (7.8.1)
  - Locking guzzlehttp/promises (2.0.2)
  - Locking guzzlehttp/psr7 (2.6.2)
  - Locking guzzlehttp/uri-template (v1.0.3)
  - Locking hamcrest/hamcrest-php (v2.0.1)
  - Locking laravel/framework (v10.48.4)
  - Locking laravel/pint (v1.15.1)
  - Locking laravel/prompts (v0.1.17)
  - Locking laravel/sail (v1.29.1)
  - Locking laravel/sanctum (v3.3.3)
  - Locking laravel/serializable-closure (v1.3.3)
  - Locking laravel/tinker (v2.9.0)
  - Locking league/commonmark (2.4.2)
  - Locking league/config (v1.2.0)
  - Locking league/flysystem (3.26.0)
  - Locking league/flysystem-local (3.25.1)
  - Locking league/mime-type-detection (1.15.0)
```

Proyecto 2º DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

Configuración de Apache para el correcto enrutamiento

```
usuario@servidor:/var/www/ccc$ sudo chmod +777 -R ccc-app/
[sudo] contraseña para usuario:
usuario@servidor:/var/www/ccc$ sudo a2enmod rewrite
Enabling module rewrite.
To activate the new configuration, you need to run:
  systemctl restart apache2
usuario@servidor:/var/www/ccc$ sudo systemctl restart apache2
```

Configuración del site de Apache

```
 GNU nano 6.2                                  /etc/apache2/sites-available/ccc.conf
<VirtualHost *:80>
        # The ServerName directive sets the request scheme, hostname and port that
        # the server uses to identify itself. This is used when creating
        # redirection URLs. In the context of virtual hosts, the ServerName
        # specifies what hostname must appear in the request's Host: header to
        # match this virtual host. For the default virtual host (this file) this
        # value is not decisive as it is used as a last resort host regardless.
        # However, you must set it for any further virtual host explicitly.
        #ServerName crimecasecentral.com

        ServerAdmin webmaster@localhost
        DocumentRoot /var/www/ccc/ccc-app/public

        # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
        # error, crit, alert, emerg.
        # It is also possible to configure the loglevel for particular
        # modules, e.g.
        #LogLevel info ssl:warn
        <Directory /var/www/ccc/ccc-app>
                Options Indexes MultiViews
                AllowOverride None
                Require all granted
        </Directory>



        ErrorLog ${APACHE_LOG_DIR}/ccc_error.log
        CustomLog ${APACHE_LOG_DIR}/ccc_access.log combined

        # For most configuration files from conf-available/, which are
        # enabled or disabled at a global level, it is possible to
        # include a line for only one particular virtual host. For example the
        # following line enables the CGI configuration for this host only
        # after it has been globally disabled with "a2disconf".
```

10. Manual de usuario

Se brindará un manual de usuario que, como se ha comentado anteriormente, facilitará a los usuarios finales el uso del aplicativo. No obstante, el plan de formación a los usuarios finales se detalla en el siguiente apartado.

11. Plan de Formación a los Usuarios de la Aplicación

Contenidos a tratar.

Objetivo:

El objetivo de este plan de formación es capacitar a los usuarios de Crime Case Central en el uso efectivo de la aplicación, asegurando que puedan aprovechar al

máximo sus funcionalidades para la gestión de información delictiva y la investigación criminal.

Duración del Plan:
El plan de formación se llevará a cabo durante un período de dos semanas, con sesiones programadas regularmente para cubrir diferentes aspectos de la aplicación.

Tiempo estimado de impartición de cada contenido.

Contenido del Plan:
Semana 1: Fundamentos de Crime Case Central

- **Día 1:** Introducción a Crime Case Central: Visión general de la aplicación, sus objetivos y funcionalidades principales.
- **Día 2:** Registro de Usuarios: Proceso de creación de cuentas, gestión de roles y permisos.
- **Día 3:** Navegación e Interfaz de Usuario: Familiarización con la interfaz de usuario de Crime Case Central, incluyendo la navegación por menús y la ubicación de funciones clave.
- **Día 4:** Gestión de Registros de Crímenes: Creación, modificación y eliminación de registros de crímenes, incluyendo la entrada de datos y la categorización adecuada.
- **Día 5:** Búsqueda y Filtrado: Uso de herramientas de búsqueda y filtrado para localizar rápidamente información relevante dentro de la aplicación.

Semana 2: Funcionalidades Avanzadas y Casos Prácticos

- **Día 6:** Colaboración y Comunicación: Uso de funciones de colaboración, como comentarios y notas, para facilitar la comunicación entre usuarios.
- **Día 7:** Análisis de Datos: Utilización de herramientas de análisis de datos para identificar patrones, tendencias y relaciones en la información delictiva.
- **Día 8:** Gestión de Casos: Organización y seguimiento de casos criminales completos dentro de Crime Case Central, incluyendo la asignación de recursos y la gestión de plazos.

Proyecto 2DAW. Autores: Ahmed Hassan Khamis y Christian Miguel Giménez Puerma

- **Día 9:** Seguridad y Privacidad: Concienciación sobre las mejores prácticas de seguridad y privacidad al utilizar Crime Case Central, incluyendo la gestión de contraseñas y la protección de datos sensibles.
- **Día 10:** Casos Prácticos y Evaluación: Aplicación de los conocimientos adquiridos a través de ejercicios prácticos y evaluación del aprendizaje.

Evaluación y Seguimiento:
Al finalizar el plan de formación, se realizará una evaluación para medir la comprensión y competencia de los usuarios en el uso de Crime Case Central. Además, se ofrecerá soporte continuo y recursos de aprendizaje adicionales para garantizar que los usuarios puedan seguir desarrollando sus habilidades y aprovechando al máximo la aplicación en sus actividades diaria

12. Bibliografía utilizada para cada contenido.

Documentación criminóloga, legislación y fondos

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

- Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales: https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673
- Ley 26/2007, de 23 de octubre, de Responsabilidad Medioambiental: https://www.boe.es/buscar/act.php?id=BOE-A-2007-18475
- Incentivos y ayudas para emprendedores y autónomos del Servicio Público de Empleo Estatal (SEPE): https://www.sepe.es/HomeSepe/autonomos/incentivos-ayudas-emprendedores-autonomos
- Clasificación tipológica en criminología: https://psicologiaymente.com/forense/tipos-crimen

Otras bibliografías (miscelánea)

- Herramienta para mostrar el código a modo de captura de pantalla CodeSnap: https://codesnap.dev/
- Iconografía utilizada en el aplicativo: https://www.flaticon.es/
- Documentación oficial de Laravel 10: https://laravel.com/docs/10.x/
- Documentación W3Schools de JavaScript Vanilla: https://www.w3schools.com/js/
- Documentación W3Schools de SQL: https://www.w3schools.com/sql/
- Documentación MDN de CSS: https://developer.mozilla.org/es/docs/Web/CSS
- Foro de ayuda al desarrollo StackOverflow: https://es.stackoverflow.com/
- Manuales de informática DigitalOcean: https://www.digitalocean.com/

- Información sobre crímenes (tanto ficticios como reales) para la demo de la aplicación:
  - https://www.abandomoviez.net/dba/fichapolicial.php?id=1277
  - https://dexter.fandom.com/
  - https://criminal-mente.es/2018/01/24/analisis-criminologico-del-caso-diana-quer/
  - https://es.wikipedia.org/wiki/Caso_B%C3%A1rcenas
  - https://www.businessinsider.es/caso-ere-andalucia-resumen-claves-sentencia-511425
  - https://es.wikipedia.org/wiki/El_Dioni
  - https://icerrajerosmadrid.com/el-robo-mas-famoso/#:~:text=Con%20nada%20m%C3%A1s%20y%20nada,a%C3%BAn%20no%20se%20ha%20resuelto.
  - https://es.wikipedia.org/wiki/Caso_de_La_Manada
  - https://www.20minutos.es/noticia/5209976/0/monstruo-amstetten-josef-fritzl-violacion-su-hija-secuestro-24-anos/

Proyecto 2º DAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma

- https://es.wikipedia.org/wiki/Ana_Orantes
- http://www.foromemoriacordoba.org/wp-content/uploads/2021/01/breve-cronica-de-la-represion-franquista.pdf
- https://historia.nationalgeographic.com.es/a/holocausto-historia-genocidio-mas-famoso-historia_20329
- https://es.wikipedia.org/wiki/ILoveYou
- https://www2.deloitte.com/es/es/pages/risk/articles/los-cinco-mayores-ciberataques-de-la-historia.html
- https://es.wikipedia.org/wiki/Ataques_ransomware_WannaCry

Proyecto 2ºDAW. Autores:
Ahmed Hassan Khamis
y Christian Miguel Giménez Puerma